Engineering Applications of Artificial Intelligence 176 (2026) 114695

Contents lists available at ScienceDirect

Engineering Applications of Artificial Intelligence

journal homepage: www.elsevier.com/locate/engappai

Research paper

Direction-aware deep policy learning for efficient capacitated arc routing

Feng Xue [a,b,1], Runze Guo [a,1], Anlong Ming [a,*], Nicu Sebe [b]

[a] Beijing University of Posts and Telecommunications, Beijing, China
[b] University of Trento, Trento, Italy

ARTICLE INFO

Keywords:
Capacitated Arc Routing Problem
Direction-aware attention model
Supervised reinforcement learning
Multi-vehicle routing
Urban road coverage

ABSTRACT

Line coverage of urban-scale routes by multiple vehicles can be formulated as the Capacitated Arc Routing Problem (CARP). Early solutions, including constructive heuristics, meta-heuristics, and exact algorithms, struggle to achieve efficient runtimes. Recently, neural network (NN)-based methods have emerged as promising alternatives for efficiently solving CARP; however, their solution quality significantly lags behind non-NN approaches. This significant gap mainly results from the inappropriate modeling and learning of *non-Euclidean graphs, traversal direction and capacity constraints*. In this paper, we introduce an NN-based solver tailored for these complexities, which significantly narrows the gap with advanced meta-heuristics while achieving much shorter runtimes. First, we propose the direction-aware attention model (DaAM) to incorporate directionality into the embedding process, facilitating more effective one-stage decision-making. Second, we design a supervised reinforcement learning scheme that involves supervised pre-training to establish a robust initial policy for subsequent reinforcement fine-tuning. It proves particularly valuable for solving CARP that has a higher complexity than the node routing problems (NRPs). Finally, a path optimization method is introduced to adjust the depot return positions within the path generated by DaAM. Experiments show that DaAM surpasses constructive heuristics and achieves decision quality comparable to state-of-the-art meta-heuristics for the first time while maintaining superior efficiency, even in large-scale CARP instances. The code and datasets are provided in the *link* and will be released on *GitHub*.

1. Introduction

With the development of smart city infrastructure and breakthroughs in perception (Fan et al., 2022; Chang et al., 2022; F. Xue et al., 2024; Xue et al., 2025) and control technologies (Lin et al., 2025; Debarshi et al., 2022; Berahman et al., 2025) in self-driving cars, the deployment of vehicle group (Huo et al., 2025; P. Xue et al., 2024; Fu et al., 2025) consisting of mobile robots and autonomous vehicles has become a foreseeable trend. These cooperative vehicles are tasked with large-scale operations such as road-surface inspection, environmental sensing, refuse collection and emergency logistics. They navigate dense urban road networks, cover every required street under restricted energy and payload constraints. Consequently, the quality and efficiency of their route planning directly affect service and operating costs.

Theoretically, an urban road network can be modeled as a connected undirected graph $\mathbf{G} = (\mathbf{V}, \mathbf{E}, \mathbf{E}_R)$, where $\mathbf{V}$ is the set of nodes, $\mathbf{E}$ is the set of street (edges), and $\mathbf{E}_R \subseteq \mathbf{E}$ comprises the serviceable streets (required edges). When several vehicles depart from a common depot $depot \in \mathbf{V}$ and each is limited by load or battery capacity, denoted as Q, the task is exactly the Capacitated Arc Routing Problem (CARP). CARP asks for visiting sequences for all required edges $\mathbf{E}_R$ while minimizing all vehicles' total travel cost under capacity constraint of each vehicle. Different from the original arc routing problem (ARP), CARP couples the sequencing of edges with the assignment of edges to vehicles, causing a coupled combinatorial explosion (Golden and Wong, 1981). This coupling is the key bottleneck that also underlies more elaborate variants—Directed, Windy, Multi-Depot, and Time-Window CARPs, etc. Consequently, CARP serves as a gateway problem for learning-based routing research and carries substantial scientific and practical value.

According to Golden and Wong (1981), CARP is NP-hard. Over the years, a variety of ***meta-heuristics***, including tabu search (Hertz et al., 2000), quantum-inspired (Shang et al., 2018), memetic (Krasnogor and Smith, 2005; Tang et al., 2009; Li et al., 2021) algorithms (MAs), etc., have been developed to obtain near-optimal solutions. The Memetic Algorithm with Extended Neighborhood Search (MAENS) (Tang et al., 2009) (an MA tailored for CARP) has been regarded as the state-of-the-art and is still the benchmark against which new methods are

* Correspondence to: Beijing University of Posts and Telecommunications, 806 New Research Building, No. 10 Xitucheng Road, Haidian District, Beijing, 100876, China.
E-mail address: mal@bupt.edu.cn (A. Ming).
[1] Equal contribution.

https://doi.org/10.1016/j.engappai.2026.114695
Received 5 September 2025; Received in revised form 25 March 2026; Accepted 31 March 2026
Available online 15 April 2026
0952-1976/© 2026 Elsevier Ltd. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

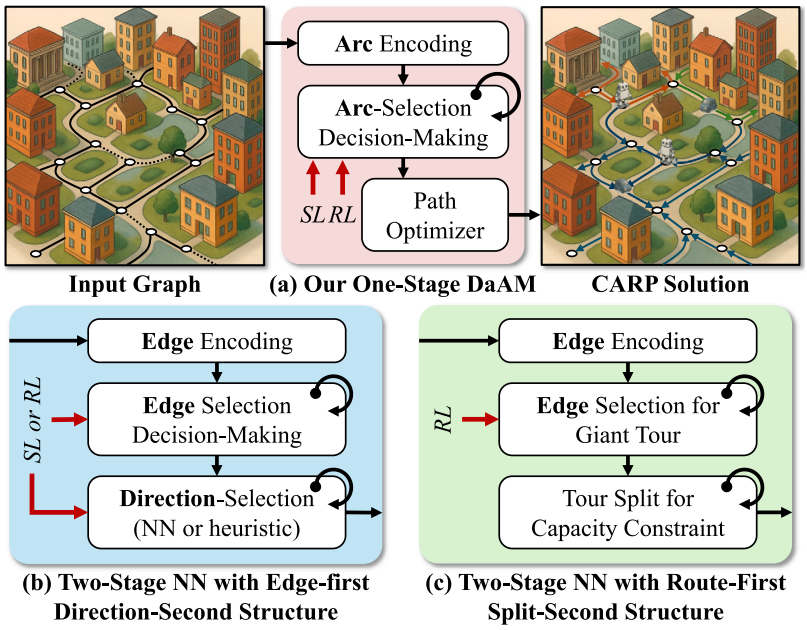


Fig. 1. Comparison of (a) our DaAM and (b–c) existing CARP neural models. In the input graph, solid lines are required edges, dotted lines are non-service edges, and circles are nodes. In the solution, colors represent the routes of different vehicles.

judged (Li and Li, 2019). However, they struggle with high time costs due to the exponential growth of the search space as the problem scale increases. Another affordable choice is ***constructive heuristics***, such as the classical path scanning family (Golden and Wong, 1981; Santos et al., 2009) and its modern variants (Arakaki and Usberti, 2019). By relying on greedy rules, they keep computation times to seconds for practical scenarios, but their solutions are always inferior to those of advanced meta-heuristics. Compared to the traditional methods, ***NN-based solvers*** (Li and Li, 2019; Hong and Liu, 2022; Ramamoorthy and Syrotiuk, 2024), as shown in Fig. 1(b) and (c), are faster with the assistance of GPU, and therefore have gained increasing attention in recent years. However, unlike the decent performance in solving NRP, such as vehicle routing problem (VRP) and traveling salesman problem (TSP), or ARP defined in Euclidean graphs, such as rural postman problem (RPP) and Chinese postman problem (CPP), NN-based methods usually lag far behind the traditional ones in solving CARP, even with a performance gap of 10%–30% (Hong and Liu, 2022; Li and Li, 2019). This discrepancy is attributed to the inability of existing methods to effectively reduce the high complexity of solving CARP:

- **Inefficient multi-stage decision-making.** In CARP, edge direction, transition cost and capacity are tightly coupled. However, existing NN-based methods (Li and Li, 2019; Hong and Liu, 2022; Ramamoorthy and Syrotiuk, 2024) break the decision process into two steps, choosing undirected edges firstly and directions secondly, which suffers from triple issues. ① The 1st step cannot measure precise transition cost, making poor decision; ② In the 2nd step, too many direction candidates for greedy search would slow computation; ③ A wrong direction might violate the capacity constraint, leaving RL with weak feedback.
- **Learning difficulty**: During reinforcement learning, a lot of candidate actions are unavailable due to the capacity constraint, making the reward signal extremely sparse. This yields high-variance policy-gradient estimates, further making convergence slow and unstable.

In this paper, we aim to address both above issues and propose an NN-based method for CARP that competes with the state-of-the-art MA (Tang et al., 2009) while with significantly lower runtime, as shown in Fig. 1(a). First, we propose a direction-aware attention model (DaAM). This model encodes direction selection, edge selection, remaining capacity, etc. into a simple and efficient single-stage decision process, which is aligned with the inherent properties of CARP. Through this design, DaAM has fewer decision steps and employ complete state information for decision making, which makes CARP solving more efficient. Second, we develop a supervised reinforcement learning approach to overcome the learning difficulty of CARP. It first imitates meta-heuristics expert policies to establish a baseline and then fine-tunes on large-scale CARP instances using proximal policy optimization with a self-critical strategy. The two-stage training achieves fast convergence on learning CARP. Finally, we observe that current NN-based CARP solvers often struggle to determine return positions with optimal cost. To further optimize the return positions, we adopt a post-processing path optimizer (PO) that is compatible with most CARP solvers and further improves path quality. In the experiments, our method approaches the state-of-the-art MA with an average gap of 5% and is 4% better than the latest constructive heuristics and gains high efficiency. Our contributions are as follows:

1. As far as we know, the proposed DaAM is the first neural model to frame the CARP as a single-stage decision-making task, addressing the inefficiency and learning difficulty prevalent in multi-stage decision frameworks.
2. We model CARP using arc tokens (with explicit direction choices) and equip the decoder with dynamic feasibility constraints (capacity and reverse-arc mutual-exclusion masks), which allows DaAM to generate a direction-aware giant arc tour efficiently and reliably.
3. Extensive experiments show that DaAM achieves solution quality closer to meta-heuristics while with faster inference than advanced constructive heuristics, establishing a strong baseline for many CARP-extended problems.

2. Related work

2.1. Graph embedding learning

Graph embedding aims to map nodes or edges in a graph to a low-dimensional vector space. This process is commonly achieved via

graph neural networks (GNNs) (Wu et al., 2020). Kipf and Welling (2017) introduced graph convolutional operations that aggregate information from neighboring nodes for updating node representations. Unlike GCN, Graph Attention Network (GAT) (Veličković et al., 2018) allowed dynamic node attention during information propagation by attention mechanisms. Other GNN variants (Hamilton et al., 2017; Wu et al., 2019) exhibited a similar information aggregation pattern but with different computational approaches. In this paper, since an arc is related to the outgoing arc of its endpoint but irrelevant to the incoming arc of that, we use attention mechanisms to capture the intricate relationships between arcs for arc embedding learning.

2.2. Learning methods for routing problems

The routing problem on graphs is a classic combinatorial optimization problem (COP) (Khalil et al., 2017), mainly divided into node and arc routing problems according to the decision element.

Node routing problems (NRPs), such as the TSP (Li and Zhang, 2024; Ebadinezhad, 2020; Duan et al., 2025) and VRP (Campuzano et al., 2025; Wang et al., 2025a), aim to find the optimal path that visits all nodes. Since the solutions to these problems are context-dependent variable-length sequences, they cannot be directly modeled using Seq2Seq methods (Sutskever et al., 2014). Vinyals et al. (2015) addressed this with the Pointer Network (PN) for the Euclidean TSP. To address label scarcity, Bello et al. (2017) reformulated the TSP as a single-step reinforcement learning problem and trained the PN with policy gradients under the A3C framework (Williams, 1992; Mnih et al., 2016). Kool et al. (2019) then employed multi-head attention for embedding learning and used REINFORCE (Williams, 1992) with a greedy baseline for training. Nazari et al. (2018) replaced the LSTM encoder in the PN with an element-wise projection layer, and used an RNN decoder with attention to sequentially select the next customer under feasibility masks while updating dynamic states (*e.g.*, remaining demand and vehicle load), pioneering a neural network approach for the Euclidean VRP and its variants. For COPs in graphs, Khalil et al. (2017) proposed S2V-DQN, which uses structure2vec (Dai et al., 2016) to learn graph embeddings and applies (n-step) DQN (Mnih et al., 2015) to iteratively construct solutions by selecting the next action based on predicted Q-values, enabling end-to-end learning without supervised labels. More recently, Liu et al. (2024) represents VRPs as combinations of shared attributes, enabling a single model to generalize across problems and boost performance. Although these NN-based methods perform comparably to meta-heuristics, they cannot be directly applied to arc routing problems due to fundamental modeling differences.

Arc routing problems (ARPs) seek optimal edge traversals in graphs, including variants such as RPP (Easton and Burdick, 2005), CARP (Golden and Wong, 1981), and the line coverage problem (Agarwal and Akella, 2024). Tran and Hy (2023) developed a DRL framework for CPP with load-dependent costs on Euclidean graphs, outperforming meta-heuristics. However, CARP is defined on non-Euclidean graphs that lack inherent node coordinates, requiring manual extraction of node representations—a task suited for learning. Although several NN-based methods have been proposed, their performance still lags behind traditional approaches. Li and Li (2019) first applied an NN-based approach to CARP by converting it into an NRP: they determined edge sequences and then set traversal directions. Hong and Liu (2022) supervised a Pointer Network to select undirected edges and determined directions in post-processing. Ramamoorthy and Syrotiuk (2024) generated an initial tour from edge embeddings before splitting it into capacity-constrained routes. However, these approaches lack proper edge direction encoding, resulting in suboptimal edge selection and necessitating a two-stage decision process or extra splitting.

2.3. Solvers for capacitated arc routing problem (CARP)

The capacitated arc routing problem (CARP) was introduced by Golden and Wong (1981) and has been extensively studied in the arc-routing literature (Corberán and Prins, 2010). Existing solvers can be broadly categorized into **exact methods**, classical **heuristics/metaheuristics**, and **learning-based approaches**. **Exact solvers** (*e.g.*, MIP and branch-and-price/branch-and-cut-and-price) can provide optimality guarantees and strong lower bounds but typically scale only to small or medium instances due to the NP-hardness of CARP (Bode and Irnich, 2012). **Constructive heuristics** (Corberán and Laporte, 2015; Labadie et al., 2016) such as path-scanning (PS) generate feasible solutions in negligible time and are often used for initialization or as fast references. Many improved PS variants have been proposed to strengthen local decision rules (Arakaki and Usberti, 2019).

Metaheuristics have long been the main workhorse for CARP, including tabu search and guided local search (Hertz et al., 2000; Beullens et al., 2003), deterministic tabu search (Brandão and Eglese, 2008), and memetic frameworks such as Memetic Algorithm with Extended Neighborhood Search (MAENS) (Tang et al., 2009) and HMA (Chen et al., 2016). Although MAENS dates back to 2009, it remains a strong and widely adopted baseline in subsequent CARP studies (Chen et al., 2016; Vidal, 2017). For large-scale CARP, newer solvers emphasize scalability via decomposition or hierarchical strategies, *e.g.*, SAHiD (Tang et al., 2017), cooperative coevolution with route-distance grouping (Mei et al., 2014), FastCARP (Wøhlk, 2018), route cutting-off divide-and-conquer decomposition (Zhang et al., 2019), and divide-and-conquer operators integrated into memetic search for large-scale mixed CARP (Oliveira et al., 2025). **Neural and learning-based solvers** have also emerged, including early DRL formulations (Li and Li, 2019), fast sequence-to-sequence inference (Hong and Liu, 2022), and more recent CARP-tailored neural solvers. AEGT (Jia et al., 2025) models the decision unit as an unserved required edge and strengthens local structural modeling via traversal-based feature representation and adjacent attention over reachable neighborhoods. DAR (Wang et al., 2025b) proposes distance-aware attention reshaping to mitigate attention-score diffusion when generalizing across scales and distributions, serving as a plug-and-play enhancement to attention-based neural solvers without introducing additional trainable parameters. Beyond CARP, Nguyen et al. (2025) study the hierarchical directed variant (HDCARP) and hybridize reinforcement learning with heuristics by using RL to adaptively select search operators online, improving efficiency under hierarchical/directed constraints.

Unlike prior neural solvers for CARP, our method represents each required edge as two directed arcs and performs one-stage direction-aware decoding at the arc level, directly incorporating directionality and arc-to-arc relational costs into the sequential decision process.

3. Background

The attention model (AM) (Kool et al., 2019) exhibits superior effectiveness in solving classic Euclidean COPs due to its attention mechanisms for extracting correlations between inputs. Therefore, we use the AM as the backbone and give a brief review in terms of the TSP. Given an Euclidean graph $\mathbf{G}$=($\mathbf{V}$, $\mathbf{E}$), the AM defines a stochastic policy, denoted as $\pi(\boldsymbol{x}|S)$, where $\boldsymbol{x} = (x_0, \ldots, x_{|\mathbf{V}|-1})$ represents a permutation of the node indexes in $\mathbf{V}$, and S is the problem instance expressing $\mathbf{G}$. The AM is parameterized by θ as:

$$\pi_\theta(\boldsymbol{x}|S) = \prod_{t=1}^{|\mathbf{V}|} \pi_\theta(x_t|S, \boldsymbol{x}_{0:t-1}) \tag{1}$$

where t denotes the time step. Specifically, the AM comprises an encoder and a decoder. The encoder first computes initial d_h-dimensional embeddings for each node in $\mathbf{V}$ as h_i^0 through a learned linear projection. It then captures the embeddings of h_i^0 using multiple attention layers, with each comprising a multi-head attention (MHA) sublayer and a node-wise feed-forward (FF) sublayer. Both types of sublayers include a skip connection and batch normalization (BN). Assuming

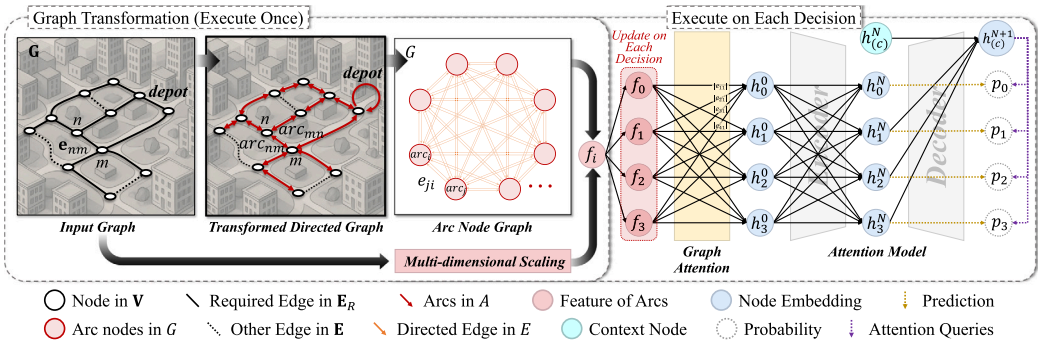


Fig. 2. DaAM pipeline consists of two parts. The first part transforms the input graph $\mathbf{G}$ by treating the arcs on $\mathbf{G}$ as nodes of a new directed graph G, only executing once. The second part leverages the GAT and AM to update arc embeddings and select arcs, executing at each time step.

that $l \in \{1, \dots, N\}$ denotes the attention layer, the lth layer can be formulated as h_i^l:

$$h_i^l = \mathrm{BN}^l(\hat{h}_i + \mathrm{FF}^l(\hat{h}_i)) \tag{2}$$

$$\hat{h}_i = \mathrm{BN}^l(h_i^{l-1} + \mathrm{MHA}_i^l(h_0^{l-1}, \dots, h_{|\mathbf{V}|-1}^{l-1})) \tag{3}$$

The decoder aims to append a node to the sequence $\boldsymbol{x}$ at each time step. Specifically, a context embedding $h_{(c)}$ is computed to represent the state at the time step t. Then a single attention head is used to calculate the probabilities for each node based on $h_{(c)}$:

$$u_{(c)j} = \begin{cases} C \cdot \tanh\left(\dfrac{[\mathbf{W}^Q h_{(c)}]^T \mathbf{W}^K h_j^N}{\sqrt{d_h}}\right) & \text{if } j \neq x_{t'} (\forall t' < t) \\ -\infty & \text{otherwise,} \end{cases}$$

$$p_i = \pi_\theta(x_t = i | S, \boldsymbol{x}_{0:t-1}) = \frac{u_{(c)i}}{\sum_j u_{(c)j}}, \tag{4}$$

where $\mathbf{W}^Q$ and $\mathbf{W}^K$ are the learnable parameters of the last attention layer. $u_{(c)j}$ is an unnormalized log probability with (c) indicating the context node. C is a constant, and p_i is the probability distribution computed by the softmax function based on $u_{(c)j}$.

4. Direction-aware attention model

Given the formulation of CARP in the introduction Section 1, we propose the direction-aware attention model (DaAM) for efficient one-stage CARP decision-making. In Section 4.1, we describe the structure of DaAM. In Section 4.2, we give the training scheme of DaAM. Finally, in Section 4.3, the path optimization, an optional post-processing unit, is introduced to improve the CARP solution quality further.

4.1. One-stage structure of direction-aware attention model

In this section, we propose the direction-aware attention model (DaAM). Unlike previous methods that learn edge embeddings and determine edge directions separately, our model encodes direction information directly into the embedding, enabling one-stage decision-making. As shown in Fig. 2, the DaAM makes sequential decisions in two phases to select arcs. **The first phase** is a one-time transformation process, in which the arcs of the input graph are represented as nodes in the new directed complete graph. **The second phase** is executed at each time step to select the arc of each action.

4.1.1. Arc feature formulation via graph transformation

Graph transformation. Our method begins from the edge-to-arc decomposition for explicit edge direction encoding. Let $\mathbf{G}$=($\mathbf{V}, \mathbf{E}, \mathbf{E}_R$) denotes the undirected connected graph as input, where $\mathbf{V}$ is the node set of $\mathbf{G}$, $\mathbf{E}$ is the edge set of $\mathbf{G}$, and $\mathbf{E}_R \subseteq \mathbf{E}$ is the required edge set. Firstly, following Vidal (2017), given that an edge has two potential traversal directions, we decompose each edge $\mathbf{e}_{nm} = (cost_{nm}, demand_{nm}, allow_serve_{nm}) \in \mathbf{E}_R$ into two arcs $\{arc_{nm}, arc_{mn}\}$ with opposite directions but the same cost, demand and serving state. Here n, m are the indexes of node in $\mathbf{V}$. To simplify the representation below, we replace nm and mn with single-word symbols, such as i and j. In this way of edge decomposition, we obtain a set of arcs denoted as A_R. Secondly, we build a new graph $G = (A_R, E)$. Specifically, each arc in A_R serves as a node in G, and directed edge set E is created, with $e_{ij} \in E$ representing the edge from node arc_i to arc_j. The weight $|e_{ij}|$ represents the total cost of the shortest path from the end node of arc_i to the start node of arc_j. In addition, we treat the depot as a self-loop zero-demand arc that allows for repeated serving, denoted as arc_0. Consequently, we transform the input graph $\mathbf{G}$ into a directed complete graph G where each node in G corresponds an arc in $\mathbf{G}$. In this way, it is natural to directly select the arcs from G during the decision-making. Given that the Floyd–Warshall algorithm is used to calculate the shortest path cost between any pair of nodes in $\mathbf{G}$, the time complexity of our graph transformation is $\max(\mathcal{O}(|\mathbf{E}_R|^2), \mathcal{O}(|\mathbf{V}|^3))$.

Table 1
Feature detail of arc_i at time step t for CARP.

No.	Symbol	Field	Description
1	is_depot_i	$\mathbb{F}_2$	Is arc_i the depot?
2	$cost_i$	$\mathbb{R}^+$	Cost of arc_i.
3	$demand_i$	$\mathbb{R}^+$	Demand of arc_i.
4	$\lvert e_{x_{t-1}i} \rvert$	$\mathbb{R}^+$	Edge weight from $arc_{x_{t-1}}$ to arc_i.
5	$serve_t^{(i)}$	$\mathbb{F}_2$	Is arc_i at time t allowed to serve?
6	$mds_{\mathrm{start}(i)}$	$\mathbb{R}^{+d}$	Euclidean start coordinates of arc_i
7	$mds_{\mathrm{end}(i)}$	$\mathbb{R}^{+d}$	Euclidean end coordinates of arc_i

Arc feature formulation. To support decision-making for arc selection, we construct arc features as input for the subsequent model. Specifically, multi-dimensional scaling (MDS) is used to project the input graph $\mathbf{G}$ into a d-dimensional Euclidean space. The Euclidean coordinates of arc_i's start and end nodes, denoted as $mds_{\mathrm{start}(i)}$ and $mds_{\mathrm{end}(i)}$, are then taken as the features of arc_i to indicate its direction. As shown in Table 1, at time step t, arc_i can be featured as:

$$F_t^{(i)} = (is_depot_i, cost_i, demand_i, \lvert e_{x_{t-1}i} \rvert, serve_t^{(i)}, mds_{\mathrm{start}(i)}, mds_{\mathrm{end}(i)}) \tag{5}$$

where x_{t-1} is the index of the selected arc at the last time step, and $t \in [1, +\infty)$, $|e_{x_{t-1}i}|$ is the transition cost from $arc_{x_{t-1}}$ to candidate arc_i in the transformed arc graph. $serve_t^{(i)} \in \{0, 1\}$ is a time-varying service-status flag indicating whether arc_i is still unserved/eligible at step t. Once an arc is served at time t, we set $serve_{t'}^{(i)} = 0$ for all $t' > t$. According to Eq. (5) and Table 1, the graph construction (transforming the original CARP instance into the arc-based complete directed graph), shortest-path costs, and MDS coordinates are computed only once. While the time-varying features, *i.e.*, $|e_{x_{t-1}i}|$ and $serve_t^{(i)}$, are computed at every decision step t together with the subsequent GAT layer and AM.

Our feature models arcs rather than edges and encodes the direction attribute of arcs through MDS. Therefore, it is more suitable than previous methods (Hong and Liu, 2022; Li and Li, 2019) for ARPs that need to consider the direction of traversing.

4.1.2. Arc relation encoding

Although AM is efficient in decision-making, according to Eq. (2), it cannot encode the edge weights between nodes in G, an important context feature, during learning. Inspired by the Graph Attention Network (GAT) (Veličković et al., 2018), we apply a layer of graph attention to fuse the dynamic arc features and the edge weights between two arcs into the initial embedding, which is for better decision of the attention model (in Section 4.1.3). Specifically, at each time step t, for each arc arc_i, we integrate the weights between arc_i and all arcs in A_R along with their features into the initial embedding of arc_i.

$$c_{ij} = softmax\big(\alpha(\mathbf{W}_{\text{att}}[\, F_t^{(i)} \,\|\, F_t^{(j)} \,\|\, |e_{ji}|\,])\big),$$
$$h_i^0 = \sigma\Big(\sum_{j=0}^{|A_R|-1} c_{ij}\mathbf{W}_{\text{emb}}F_t^{(j)}\Big), \tag{6}$$

where $\mathbf{W}_{\text{att}} \in \mathbb{R}^{1\times 43}$ and $\mathbf{W}_{\text{emb}} \in \mathbb{R}^{d_h\times 21}$ are learnable parameters, and d_h is the dimension of feature embeddings. $[\cdot \,\|\, \cdot]$ is the horizontal concatenation operator, $\alpha(\cdot)$ is a mapping from the input to a scalar, and $\sigma(\cdot)$ denotes the activation function. h_i^0 denotes the initial feature embedding of arc_i, which is taken as the input of subsequent attention model. Note that we use only one graph attention layer to avoid over-smoothing and efficiency issues (Chen et al., 2020) suffered from deep GAT.

4.1.3. Arc selection

After aggregating the edge weights of G into the initial embeddings, we utilize attention model (AM) to learn the final arc embeddings and make arc selection decisions. In the encoding phase described by Eq. (2), for each arc $\{arc_i\}$, we leverage N attention layers to process the initial embeddings $\{h_i^0\}$ and obtain the output embeddings of the Nth layer, *i.e.*, $\{h_i^N\}$. In the decoding phase, the context node applicable to CARP can be defined as:

$$h_{(c)}^N = \Big[\frac{1}{|A_R|}\sum_{i=0}^{|A_R|-1} h_i^N, h_{x_{t-1}}^N, \delta_t, \Delta_t\Big], t \in [1, +\infty) \tag{7}$$

where x_{t-1} indicates the chosen arc index at time step $t-1$ and x_0 is arc_0. δ_t is the remaining capacity at time step t, $\Delta_t = \Delta(\delta_t > \frac{Q}{2})$ is a variable to indicate whether the vehicle's remaining capacity exceeds half. Finally, according to Eq. (7), the decoder of AM takes the context node $h_{(c)}^N$ and arc embeddings $\{h_i^N\}$ as inputs and calculates the probabilities for all arcs, denoted as p_i. The serviceable arc selected at time step t, *i.e.*, arc_{x_t}, is determined by sampling or greedy decoding.

Masking for feasibility and served edges. At each decision step t, we maintain a binary feasibility mask $\mathbf{M}_t \in \{0,1\}^{|A_R|}$, where $\mathbf{M}_t(i) = 1$ means arc i is forbidden at step t. This online-updated mask is applied directly in the decoder (masked arcs are assigned zero probability) with three purposes:

1. **Served-edge masking.** Once an arc arc_{x_t} is selected, the service flag in its feature, *i.e.*, $serve_t^{(i)}$, is set to 0 so that it remains masked in all subsequent steps.
2. **Opposite-arc mutual exclusion.** Assuming that two directed arcs for an undirected required edge $\{n, m\}$ are denoted as arc_{nm} and arc_{mn}, once one direction is selected (*e.g.*, arc_{nm} at step t), both arc_{nm} and its reverse arc_{mn} are immediately masked out for all subsequent steps. Equivalently, their service flag $serve_{t'}^{(i)}$ are set to 0 for $t' > t$.
3. **Capacity-feasibility masking.** At each time t, we track the remaining capacity δ_t in Eq. (7) for the current route. All candidate arc i whose demand exceeds δ_t is masked out in $\mathbf{M}_t$. After selecting a service arc, δ_t is decreased by its demand. Once selecting the depot arc arc_0 (*i.e.*, starting a new route), δ_t is reset to Q.

Overall, the mask $\mathbf{M}_t$ is obtained by combining the above three conditions at each time step, guaranteeing that the decoder only selects arcs that are unserved, non-conflicting, and capacity-feasible.

4.2. Supervised reinforcement learning for DaAM

The decision-making of selecting arcs can be modeled as a Markov decision process with the following symbols regarding reinforcement learning:

- **State** s_t is the newest path of arcs selected from G: $(arc_{x_0}, \ldots, arc_{x_{t-1}})$, while the terminal state is s_T with T indicating the final time step.
- **Action** a_t is the selected arc at time step t, *i.e.*, arc_{x_t}. Selecting the action a_t would add arc_{x_t} to the end of the current path s_t and tag the corresponding arcs of arc_{x_t} with their features *allow_serve* changed to 0. Notably, arc_0 can be selected repeatedly but not consecutively.
- **Reward** r_t is obtained after taking action a_t at state s_t, which equals the negative shortest path cost from the last arc $arc_{x_{t-1}}$ to the selected arc arc_{x_t}.
- **Stochastic policy** $\pi(a_t|s_t)$ specifies the probability distribution over all actions at state s_t.

We parameterize the stochastic policy of DaAM with θ:

$$\pi(x_t|\, \mathcal{S}, \boldsymbol{x}_{0:t-1}) = \pi_\theta(a_t|s_t) \tag{8}$$

where $\mathcal{S}$ is a CARP instance. Starting from initial state s_0, we get a trajectory $\tau = (s_0, a_0, r_0, \ldots, r_{T-1}, s_T)$ using π_θ. The goal of learning is to maximize the cumulative reward: $R(\tau) = \sum_{t=0}^{T-1} r_t$. However, due to the high complexity of CARP, vanilla deep reinforcement learning methods learn feasible strategies inefficiently. A natural solution is to minimize the difference between the model's decisions and expert decisions. To achieve this, we employ supervised learning to learn an initial policy based on labeled data and then fine-tune the model through reinforcement learning.

Algorithm 1 PPO algorithm with self-critical strategy

Input: batch size B, number of episodes K, training instances $\mathcal{P}$, test instances $\mathcal{T}$

Initialize policies $\pi_\theta, \pi_b \leftarrow \pi_s$

for episode $k = 1$ to K **do**
  Initialize data batch $\mathcal{M}, \mathcal{M}' \leftarrow ()$
  while $|\mathcal{M}| < B$ **do**
    Sample a CARP instance $\mathcal{S}$ from $\mathcal{P}$
    Sample $\tau_o = (s_0, a_0, \ldots, s_T)$ from $\mathcal{S}$ using π_b
    $\mathcal{M} \leftarrow \mathcal{M} \cup \{(s_0, a_0), \ldots, (s_{T-1}, a_{T-1})\}$
  end while
  for each $(s, a) \in \mathcal{M}$ **do**
    Generate trajectory τ_s^θ using π_θ from s by sampling
    Generate trajectory τ_s^b using π_b from s by greedy decoding
    Compute advantage $\mathcal{A}_s = R(\tau_s^\theta) - R(\tau_s^b)$
    $\mathcal{M}' \leftarrow \mathcal{M}' \cup \{(s, a, \mathcal{A}_s)\}$
  end for
  Update π_θ using Adam over (10) based on $\mathcal{M}'$
  if π_θ outperforms π_b on $\mathcal{T}$ **then**
    $\pi_b \leftarrow \pi_\theta$
  end if
end for

4.2.1. Supervised pre-training via multi-class classification

In the pre-training stage, we consider arc-selection at each time step as a multi-class classification task, and employ the state-of-the-art CARP

method MAENS to obtain high-quality paths as the label. Assuming that $y_t \in \mathbb{R}^{|A_R|}$ denotes the one-hot label vector at time step t of any path, with $y_t^{(k)}$ indicating each element. We utilize the cross-entropy loss to train the policy represented in Eq. (8):

$$L = -\sum_{t=0}^{T-1}\sum_{k=0}^{|A_R|-1} y_t^{(k)} \log\left(\pi_\theta(arc_k|s_t)\right) \tag{9}$$

We use the policy optimized by cross-entropy, denoted as π_s, to initialize the policy network π_θ and as the baseline policy π_b in reinforcement learning.

4.2.2. Reinforcement fine-tuning via PPO with self-critical strategy

During the fine-tuning phase, we use Proximal Policy Optimization (PPO) to optimize our model $\pi_\theta(a_t|s_t)$ due to its outstanding stability in policy updates. Given the low sample efficiency in reinforcement learning, we employ a training approach similar to self-critical training (Rennie et al., 2017) to reduce gradient variance and expedite convergence. Specifically, as shown in Algorithm 1, we use another policy π_b to generate a trajectory and calculate its cumulative reward, serving as a baseline function. Our optimization objective is based on PPO-Clip (Schulman et al., 2017):

$$\mathbb{E}_{(s,a)\sim\pi_b}\Bigg[\min\left(\frac{\pi_\theta(a|s)}{\pi_b(a|s)}\left(R(\tau_s^\theta)-R(\tau_s^b)\right),\right.$$
$$\left.\text{clip}\left(\frac{\pi_\theta(a|s)}{\pi_b(a|s)}, 1-\epsilon, 1+\epsilon\right)\left(R(\tau_s^\theta)-R(\tau_s^b)\right)\right)\Bigg], \tag{10}$$

where s is used to replace current state s_t for symbol simplification, and a for a_t. $\text{clip}(w, v_{\min}, v_{\max})$ denotes constraining w within the range $[v_{\min}, v_{\max}]$, and ϵ is a hyper-parameter. τ_s^θ denotes a trajectory sampled by π_θ with s as the initial state, while τ_s^b for the trajectory greedily decoded by π_b. In greedy decoding, the action with the maximum probability is selected at each step. $R(\tau_s^\theta)-R(\tau_s^b)$ serves as an advantage measure, quantifying the advantage of the current policy π_θ compared to π_b. We maximize Eq. (10) through gradient descent, forcing the model to select actions that yield higher advantages. The baseline policy's parameters are updated if π_θ outperforms π_b.

Convergence analysis. During training, the clip term constrains the likelihood ratio between the updated policy π_θ and the baseline policy π_b to $[1-\epsilon, 1+\epsilon]$, which bounds their KL divergence according to Schulman et al. (2017). This bound ensures that any degradation in expected return is at most a second-order term, thus yielding near-monotonic policy improvement and preventing sudden performance drops after an update. In addition, this training strategy generates a greedy rollout under the current policy and take its return as a self-critical baseline. This approach not only keeps the gradient unbiased, but also significantly reduce the variance of the advantage estimate $R(\tau_s^\theta) - R(\tau_s^b)$. The clip constraint and the low-variance advantage estimator jointly suppresses policy collapse and expedites convergence, thereby improving sample efficiency.

4.3. Path optimization via dynamic programming

The complexity of the problem is heightened by the increasing capacity constraint, making it challenging for the neural network to make accurate decisions regarding the depot return positions. In this section, following the classical *route-first, split-second* strategy (*i.e.*, the *Split* algorithm) for CARP (Ulusoy, 1985), we apply a dynamic programming (DP) procedure to optimize the depot return positions by splitting a decoded giant path into capacity-feasible routes.

Given the generated path $\boldsymbol{P}$ with terminal state $s_T = (arc_{x_0}, arc_{x_1}, \dots, arc_{x_{T-1}})$, we remove all depot arcs and obtain a giant path $\boldsymbol{P}' = (arc_{x'_0}, arc_{x'_1}, \dots, arc_{x'_{T'-1}})$. The goal of PO is to insert several depot arcs arc_0 into $\boldsymbol{P}'$, so that each sub-path between two consecutive depot visits satisfies the capacity constraint, while minimizing the overall cost increase. To explicitly describe the iterative splitting process, we define $f(\boldsymbol{P}'_{0:i})$ as the minimum *increasing cost* caused by inserting depot arcs into the prefix $\boldsymbol{P}'_{0:i} = (arc_{x'_0}, \dots, arc_{x'_i})$, such that every resulting sub-path is capacity-feasible. Then the DP transition is

$$f(\boldsymbol{P}'_{0:i}) = \min_{\substack{-1\le k<i \\ \sum_{j=k+1}^{i} demand_{x'_j} \le Q}} \left(f(\boldsymbol{P}'_{0:k}) + \Delta_k\right), \qquad f(\boldsymbol{P}'_{0:-1}) = 0, \tag{11}$$

where Δ_k is the cost increase of inserting a depot return right after $arc_{x'_k}$ (*i.e.*, between $arc_{x'_k}$ and $arc_{x'_{k+1}}$):

$$\Delta_k = \begin{cases} 0, & k = -1, \\ SC(arc_{x'_k}, arc_0) + SC(arc_0, arc_{x'_{k+1}}) - SC(arc_{x'_k}, arc_{x'_{k+1}}), & 0 \le k < T'-1. \end{cases} \tag{12}$$

Eqs. (11)–(12) are equivalent to the classical *Split* DP (Ulusoy, 1985), which finds the optimal depot-insertion (route-partition) positions for a fixed giant tour under capacity constraints. By enumerating the last split position k for each prefix, Eq. (11) iteratively partitions $\boldsymbol{P}'$ into multiple capacity-feasible sub-paths (routes). The optimal increasing cost for the whole giant path is $f(\boldsymbol{P}') = f(\boldsymbol{P}'_{0:T'-1})$, and the split positions (*i.e.*, where arc_0 is inserted) are recovered by backtracking the argmin in Eq. (11), which also determines the number of routes (vehicles) automatically. The final minimum cost for path $\boldsymbol{P}$ is $f(\boldsymbol{P}')+g(\boldsymbol{P}')$, where $g(\boldsymbol{P}')$ is the unoptimized cost of $\boldsymbol{P}'$. Since $\boldsymbol{P}'$ includes only the required edges, *i.e.*, $T' = |\mathbf{E}_R|$, the time complexity of DP is $\mathcal{O}(|\mathbf{E}_R|^2)$. During path optimization, we use beam search to generate two paths with the trained policy, one with capacity-constrained and one without. Both paths are optimized using DP and the one with the minimum cost is selected as the final result. We emphasize that the *Split* DP itself is a standard component (Ulusoy, 1985), while our contribution lies in learning an *arc-level* giant path with one-stage direction decisions and integrating *Split* as a lightweight post-optimizer to improve solution quality with negligible overhead.

5. Experiments

5.1. Datasets and setting details

Beijing-CARP benchmark. To our knowledge, public CARP datasets with separate training and testing splits are lacking, which obstructs evaluation of deep learning methods. Therefore, we construct a Beijing-CARP benchmark for deep learning methods. We extracted sub-graphs from the roadmap of Beijing, China, obtained from OpenStreetMap (Haklay and Weber, 2008), to generate CARP instances for both training and testing. All instances are divided into seven datasets, each representing different problem scales, as presented in Table 2. Each dataset consists of 30,000 instances, divided into two **disjoint** subsets: 20,000 instances for training and the remaining for testing. For each instance, the vehicle capacity is set to 100.

Public datasets. Considering that DaAM is based on deep learning, totally different from most constructive heuristic or meta-heuristic algorithms, we also employ the *gdb* (Golden et al., 1983) and *Eglese* (Li and Eglese, 1996) datasets to further verify the generalization of this method on public datasets. Both datasets have been widely used in mainstream methods over the past 20 years.

Details of implementation. DaAM is implemented using the PyTorch framework and is trained on a single NVIDIA RTX 3090 GPU. The constructive heuristics and meta-heuristic methods are evaluated on an Intel Core i9-7920X with 24 cores and a CPU frequency of 4.4 GHz. We optimize the model using Adam Optimizer (Kingma and Ba, 2015). The dimension of MDS coordinates d is set to 8, and the learning rate is set to $1e^{-4}$. We set ϵ in the PPO training at 0.1. In particular, our PPO training does not incorporate discounted cumulative rewards, *i.e.*, γ is set to 1. In addition, for MDS, we use NetworkX (loyd_warshall_numpy). We then apply metric MDS with precomputed

Table 2
Beijing-CARP dataset information. $|\mathbf{V}|$ is the number of nodes, $|\mathbf{E}_R|$ is the number of required edges. *demand* represents the demand range for each required edge. Each dataset has 20,000 training instances and 10,000 test instances.

Scale	CARP instances	$\|V\|$	$\|E_R\|$	$\|E\|$	Demand range	Mean demand
Small and Medium	Task 20	25–30	20	[24,45]	[116, 162]	139.93
	Task 30	30–35	30	[34,63]	[180, 237]	209.93
	Task 40	45–50	40	[45,80]	[239, 316]	280.04
	Task 50	55–60	50	[55,99]	[311, 391]	350.01
	Task 60	65–70	60	[66,114]	[380, 466]	419.99
	Task 80	85–90	80	[89,152]	[504, 615]	559.97
	Task 100	105–110	100	[114,187]	[647, 758]	699.96
Large	Task 200	205–210	200	[234,367]	[1310, 1478]	1400.05
	Task 300	305–310	300	[359,541]	[2005, 2200]	2100.04
	Task 400	405–410	400	[492,703]	[2684, 2908]	2800.20
	Task 500	505–510	500	[624,876]	[3379, 3616]	3500.15
	Task 600	605–610	600	[762,1046]	[4066, 4324]	4199.75

dissimilarities using "`sklearn.manifold.MDS`" with `n_components=8`, `dissimilarity=precomputed`, `random_state=43`, `max_iter=300`,
and `nor-malized_stress=auto`.

During training, we incrementally add supervised pre-training (SL), reinforcement learning fine-tuning (RL), and path optimization (PO) to assess the effectiveness of our training scheme and optimization, respectively. PO is taken as the inference-time post-processing. Due to the lengthy computation times of MAENS on larger-scale datasets, SL was only performed on Task20, Task30, and Task40. The batch size for SL is set to 128. During the RL stage, greedy decoding is used to generate solutions, and except for the Task20 dataset, we utilize the training results obtained from the preceding smaller-scale dataset to initialize the model. The beam width in the PO stage is set to 2. For each dataset, we compare the mean cost of different methods on 10,000 problem instances.

Details of compared methods. We compare our method against various Path-Scanning algorithms (PS) (Golden et al., 1983; Santos et al., 2009; Arakaki and Usberti, 2019) and adapted NN-based baselines. We additionally include AEGT and DAR as recent CARP-specific neural baselines, so that DaAM is compared not only with VRP-to-CARP adaptations but also with methods originally designed for CARP. Due to the absence of publicly available code for prior NN-based CARP methods, we modify two NN-based NRP solvers to suit CARP, *i.e.* S2V-DQN (Khalil et al., 2017) and VRP-DL (Nazari et al., 2018). For consistency and fairness, we adapt S2V-DQN (Khalil et al., 2017) and VRP-DL (Nazari et al., 2018) to CARP using the same arc-based representation as DaAM: we transform each required undirected edge into two opposite directed arcs and build a fully connected arc-to-arc graph. Each method (S2V-DQN*, VRP-DL*, and DaAM) takes the same arc-node features (*e.g.*, demand/cost, depot distance, and MDS start/end embeddings) and uses the same online feasibility masking to enforce (i) served-arc and opposite-arc exclusion, and (ii) capacity feasibility based on the remaining capacity. For CARP-specific neural baselines, AEGT models the decision unit as an unserved required edge and assigns traversal directions in a post-processing DP step, whereas DAR improves attention-based neural routing by reshaping attention scores with distance/structure priors. We train and evaluate AEGT and DAR under the same training/testing protocol as DaAM in the corresponding benchmark experiments. In addition, for S2V-DQN, we first re-define the state/action to match CARP (arc-based decisions and feasibility masking). Then, we replace the original S2V message-passing encoder with GAT only, because an off-the-shelf S2V implementation for our arc-level complete graph and feature format was not available, and the original S2V module is tightly coupled to its VRP-specific state representation. Importantly, this replacement is an encoder substitution to make the baseline runnable under the CARP formulation, rather than adopting our full method. Considering that the performance of learning-based solvers like VRP-DL and S2V-DQN heavily depends on the training duration, we ensure these models are adequately trained as follows. For VRP-DL and S2V-DQN, we first ensure that both models are trained until convergence, so that they are adequately trained for fairness. As a result, the training time of S2V-DQN on different instance sizes is 0.5 h, 1.5 h, 2.5 h, 3.5 h, and 6 h respectively, and the training times of VRP-DL on different instance scales are 1 to 2 h.

For parameterization of the PS methods, we apply the official setting. For parameter α in Arakaki and Usberti (2019), PS-Ellipse's α is 1.5, PS-Efficiency's α is 3.0, PS-Alt1's α is 3.0, and PS-Alt2's α is 1.0. In terms of iteration number, the paper (Arakaki and Usberti, 2019) stated that "*For all sets of instances, the average deviations obtained by PS-Efficiency and PS-Ellipse with k = 1000 are better than those obtained by PS-RC and PS-RE with k = 20,000*". Based on this, we employ 1000 iterations for smaller problem instances from Task20 to Task100. Therefore, on smaller-scale instances, the parameters we adopted for the PS variants are exactly the same as those in the paper (Arakaki and Usberti, 2019). For larger problem instances from Task200 to Task600, we adjust the iterations to 100 to balance running time and solution quality.

Metrics. For each method and dataset, we compute the average tour cost across all test instances, indicated by "**Cost**". Following other methods (Jia et al., 2025), we take the MAENS (Tang et al., 2009) as a baseline, and measure the "**Cost**" gap between alternative algorithms and MAENS, indicated by "**Gap**" as follows:

$$\text{Gap}[\%] = \frac{\text{Cost}_{\text{method}} - \text{Cost}_{\text{MAENS}}}{\text{Cost}_{\text{MAENS}}} \times 100 \tag{13}$$

5.2. Quantitative analysis

Solution quality analysis on Beijing-CARP. Table 3 shows the results in the Beijing-CARP dataset. In this table, we also ablate the post-optimizer (PO) by comparing DaAM (SL+RL) and DaAM (SL+RL+PO), where PO is the only difference. This directly quantifies the impact of the Split-based post-optimization on solution quality. Our algorithm outperforms all constructive heuristic and NN-based methods on all scales, achieving costs comparable to MAENS, trailing by less than 8%. The advantage over PS demonstrates that neural networks can learn more effective policies than hand-crafted ones, attributed to our well-designed modeling approach. Moreover, as the problem scale increases, the time consumption of obtaining CARP annotation by MAENS is unacceptable. Therefore, instead of supervised learning on Task50, Task60, Task80, and Task100, we leverage the model pre-trained on small-scale instances as the initial policy for RL fine-tuning. This continues the advantage over PS models. Although DaAM outperforms PS variants on most scales without PO, DaAM's advantage is further widened with the help of PO. This demonstrates the generalization of our training scheme across varying problem scales. In addition, DAR and AEGT consistently underperform DaAM (SL+RL) and DaAM (SL+RL+PO), and the performance gap widens as the instance size increases, indicating superior scalability and generalization of DaAM.

Table 3
Quantitative comparison on the Beijing-CARP benchmark. All methods are evaluated on 10,000 CARP instances per scale. The gap [%] indicated in gray is measured relative to MAENS over *full precision cost*. Methods marked with an asterisk were originally designed for NRP and adapted for CARP. Bold values denote the best results. DaAM (SL+RL) vs. DaAM (SL+RL+PO) isolates the contribution of the post-optimizer (PO). @T(SL+RL) means aligning time limitation with DaAM(SL+RL). The lower bound and MIP solver are computed using Gurobi, with a time limit of 50 s. MAENS-based relative gaps are used for large-scale comparison. MIP-derived lower bounds on small scales are used as an auxiliary, more rigorous reference.

Method	Task20		Task30		Task40		Task50		Task60		Task80		Task100	
	Cost	Gap [%]	Cost	Gap [%]	Cost	Gap [%]	Cost	Gap [%]	Cost	Gap [%]	Cost	Gap [%]	Cost	Gap [%]
Lower Bound	401	–	579	–	750	–	–	–	–	–	–	–	–	–
Time-limited MIP solver	475	–	708	–	968	–	–	–	–	–	–	–	–	–
MAENS (Tang et al., 2009)	474	0	706	0	950	0	1222	0	1529	0	2113	0	2757	0
MAENS@T(SL+RL)	514	8.44	758	7.37	999	5.16	1302	6.55	1654	8.18	2254	6.67	2979	8.05
MAENS@T(SL+RL+PO)	478	0.84	717	1.56	968	1.89	1254	2.62	1585	3.66	2194	3.83	2897	5.08
HGS-CARP (Vidal, 2017)	473	−0.21	704	−0.28	948	−0.21	1220	−0.16	1525	−0.26	2104	−0.43	2738	−0.69
PS (Golden et al., 1983)	544	14.72	859	21.76	1079	13.56	1448	18.45	1879	22.84	2504	18.49	3361	21.90
PS-Ellipse (Santos et al., 2009)	519	9.49	798	13.03	1006	5.89	1328	8.67	1709	11.77	2299	8.80	3095	12.26
PS-Efficiency (Arakaki and Usberti, 2019)	514	8.44	790	11.90	1007	6.00	1311	7.28	1684	10.14	2282	8.00	3056	10.85
PS-Alt1 (Arakaki and Usberti, 2019)	514	8.44	791	12.04	1007	6.00	1312	7.36	1685	10.20	2283	8.04	3057	10.88
PS-Alt2 (Arakaki and Usberti, 2019)	521	9.92	802	13.60	1009	6.21	1336	9.33	1720	12.49	2314	9.51	3102	12.51
S2V-DQN* (Khalil et al., 2017)	590	24.42	880	24.65	1197	26.02	1520	24.32	1900	24.23	2820	33.43	3404	23.42
VRP-DL* (Nazari et al., 2018)	528	11.39	848	20.11	1193	25.57	1587	29.87	2033	32.96	2898	37.15	3867	40.26
AEGT (Jia et al., 2025)	505	6.54	770	9.07	1071	12.74	1389	13.67	1788	16.94	2532	19.83	3258	18.17
DAR (Wang et al., 2025b)	520	9.7	804	13.88	1110	16.84	1458	19.31	1864	21.91	2721	28.77	3444	24.92
DaAM (SL)	509	7.43	785	11.18	1066	12.24	–	–	–	–	–	–	–	–
DaAM (SL+RL)	495	4.48	741	5.05	1009	6.19	1303	6.58	1639	7.16	2275	7.67	2980	8.06
DaAM (SL+RL+PO)	**482**	**1.65**	**725**	**2.73**	**992**	**4.39**	**1283**	**5.07**	**1621**	**5.98**	**2255**	**6.70**	**2958**	**7.28**

Table 4
Large-scale generalization under same generator. All methods are evaluated on 10,000 CARP instances per scale. For DaAM, we use the policy trained on Task100. Bold indicates the best results, and underlined values denote the second-best.

Method	Task200 Cost	Task300 Cost	Task400 Cost	Task500 Cost	Task600 Cost
PS-Ellipse (Santos et al., 2009)	4240	6563	8600	10 909	13 377
PS-Efficiency (Arakaki and Usberti, 2019)	4233	6544	8583	<u>10 883</u>	<u>13 338</u>
PS-Alt1 (Arakaki and Usberti, 2019)	4233	6544	<u>8580</u>	10 884	13 338
PS-Alt2 (Arakaki and Usberti, 2019)	4244	6569	8606	10 922	13 393
AEGT (Jia et al., 2025) (Trained on Task100)	8001	14 783	22 460	30 704	40 072
AEGT (Jia et al., 2025) (Trained on Task200)	5794	9383	13 361	18 183	25 042
DAR (Wang et al., 2025b) (Trained on Task100)	5605	9180	13 087	17 308	22 012
DAR (Wang et al., 2025b) (Trained on Task200)	5257	8393	11 654	15 241	19 159
DaAM (SL+RL)	<u>4189</u>	<u>6372</u>	8610	10 938	13 340
DaAM (SL+RL+PO)	**4132**	**6281**	**8473**	**10 633**	**13 100**

Our algorithm outperforms other CARP-solving approaches. Note that all these results are gained under the condition of the faster speed of DaAM, which is specifically demonstrated in Section 5.3.

Ability to generalize to large-scale instances on Beijing-CARP. In Table 4, we assess DaAM's generalization on large-scale CARP instances using the policy trained on Task100. We remove MAENS and PS due to failing to run on large-scale instances, and remove S2V-DQN and VRP-DL due to poor performance. Although DaAM is not trained on large-scale instances, it achieves or even exceeds the performance of PS, which shows its potential application on larger-scale CARP instances. In extremely-large instances, such as Task 500 and Task 600, which are even **5** and **6** times the size of our training set, DaAM lags behind PS models. However, PO greatly compensates for the weakened large-instance generalization of DaAM, keep the higher practicality of our overall solution compared with PS models. In addition, AEGT and DAR exhibit clear out-of-distribution degradation when trained on Task100, but improve noticeably after retraining on Task200. In contrast, DaAM trained on Task100 remains more robust on large-scale instances, demonstrating stronger generalization than existing learning-based CARP solvers.

Quantitative analysis on public datasets. We evaluate DaAM on both the *gdb* and *Eglese* benchmarks against representative meta-heuristic, constructive-heuristic, and learning-based baselines. These baselines include, among others, PS-RC (Pearn, 1989), PS-RE (Belenguer et al., 2006), POMO (Kwon et al., 2020), MatNet (Kwon et al., 2021), and P-Reg (Sun et al., 2024) on *gdb*, as well as MA-ABC (Ramamoorthy et al., 2021) on *Eglese*. For a fair comparison with recent neural solvers, we follow AEGT and train DaAM on randomly generated road networks. At test time, DaAM uses the same POMO setting as AEGT on *gdb*, while on *Eglese* we use a fixed POMO size of 64 due to the substantially larger number of required edges. The results are reported in Tables 5 and 6. On *gdb*, DaAM outperforms the strongest learning-based baseline, AEGT, with a 0.05% difference in average gap (2.00% vs. 1.95%). On *Eglese*, DaAM clearly outperforms the other neural baselines, reducing the average gap from 24.83% for AEGT and 134.68% for DAR to 8.41%. At the same time, several path-scanning variants, especially PS-Eff. and PS-A1, remain slightly better on this benchmark. We believe this behavior is reasonable because *gdb* consists of randomly generated instances, whereas *Eglese* contains real-world winter gritting routes, which may induce a distribution shift relative to the training data. Overall, these results show that DaAM remains highly competitive among learning-based CARP solvers and generalizes much better than prior neural methods to larger real-world instances, although a small gap to the strongest handcrafted heuristics still exists on *Eglese*.

Ablation of MDS and GAT. To evaluate the combination of MDS and GAT for embedding exhibiting, we individually evaluate the performance of models using only MDS or GAT, as well as their combined performance. The variants are as follows:

Table 5
Comparison of solution costs (black) and gap [%] (blue) on the *gdb* benchmark instances. Lower values indicate better performance. For each row, the best and second-best results are highlighted in **bold** and underline, respectively, excluding *LB* and *MAENS* from ranking. LB denotes the lower bound; PS denotes path-scanning; PS-RC, PS-RE, PS-Ell., and PS-Eff. denote its random-criterion, random-edge, ellipse, and efficiency variants, respectively; and P-Reg. denotes POMO-Regret.

Inst.	LB	MAENS Tang et al. (2009)	Constructive Heuristic					Learning-based					
			PS Golden et al. (1983)	PS-RC Pearn (1989)	PS-RE Belenguer et al. (2006)	PS-Ell. Santos et al. (2009)	PS-Eff. Arakaki and Usberti (2019)	POMO Kwon et al. (2020)	MatNet Kwon et al. (2021)	P-Reg. Sun et al. (2024)	AEGT Jia et al. (2025)	DAR Wang et al. (2025b)	DaAM
gdb-1	316	316 0.00	380.2 20.32	325.9 3.13	320.2 1.33	317.3 0.41	319.8 1.20	327 3.48	**316 0.00**	330 4.43	**316 0.00**	520 64.56	**316 0.00**
gdb-2	339	339 0.00	400.7 18.20	369.9 9.12	357.3 5.40	350.6 3.42	347.7 2.57	345 1.77	347 2.36	359 5.90	**339 0.00**	509 50.15	345 1.77
gdb-3	275	275 0.00	365.5 32.91	295.1 7.31	286.9 4.33	281.5 2.36	280.9 2.15	281 2.18	286 4.00	312 13.45	**275 0.00**	457 66.18	279 1.45
gdb-4	287	287 0.00	357 24.39	295.8 3.07	288.9 0.66	289 0.70	287.4 0.14	293 2.09	**287 0.00**	308 7.32	**287 0.00**	462 60.98	**287 0.00**
gdb-5	377	377 0.00	474 25.73	403.4 7.00	396.4 5.15	383.8 1.80	383.9 1.83	386 2.39	391 3.71	402 6.63	**383 1.59**	521 38.20	395 4.77
gdb-6	298	298 0.00	375 25.84	324.1 8.76	315.5 5.87	310.6 4.23	307.5 3.19	310 4.03	**298 0.00**	323 8.39	316 6.04	530 77.85	**298 0.00**
gdb-7	325	325 0.00	398.5 22.62	337.9 3.97	328.9 1.20	326.5 0.46	325.5 0.15	334 2.77	330 1.54	347 6.77	**325 0.00**	594 82.77	329 1.23
gdb-8	348	349 0.29	465.4 33.74	416.1 19.57	414.6 19.14	371.1 6.64	365.5 5.03	382 9.77	376 8.05	362 4.02	**358 2.87**	433 24.43	377 8.33
gdb-9	303	306.3 1.09	409.6 35.18	362.9 19.77	357.7 18.05	336.7 11.12	334 10.23	338 11.55	331 9.24	334 10.23	**311 2.64**	581 91.75	342 12.87
gdb-10	275	275 0.00	325 18.18	287.3 4.47	288.9 5.05	**279.3 1.56**	280.8 2.11	292 6.18	283 2.91	319 16.00	285 3.64	379 37.82	282 2.55
gdb-11	395	395 0.00	458 15.95	445.3 12.73	437.7 10.81	412.2 4.35	411.2 4.10	447 13.16	421 6.58	435 10.13	407 3.04	530 34.18	**399 1.01**
gdb-12	458	458 0.00	581 26.86	559.4 22.14	573 25.11	499.8 9.13	495.9 8.28	503 9.83	502 9.61	510 11.35	468 2.18	907 98.03	**458 0.00**
gdb-13	536	542.9 1.29	594 10.82	563 5.04	558.9 4.27	551.7 2.93	549.2 2.46	554 3.36	**548 2.24**	588 9.70	559 4.29	758 41.42	**548 2.24**
gdb-14	100	100 0.00	125 25.00	105.9 5.90	106.6 6.60	101.6 1.60	**100 0.00**	107 7.00	104 4.00	**100 0.00**	**100 0.00**	133 33.00	**100 0.00**
gdb-15	58	58 0.00	64 10.34	60 3.45	59 1.72	59 1.72	**58 0.00**	72 24.14	64 10.34	70 20.69	**58 0.00**	70 20.69	**58 0.00**
gdb-16	127	127 0.00	139 9.45	130.6 2.83	129.8 2.20	**129 1.57**	129.4 1.89	137 7.87	133 4.72	135 6.30	131 3.15	163 28.35	**129 1.57**
gdb-17	91	91 0.00	95.2 4.62	**91 0.00**	**91 0.00**	**91 0.00**	**91 0.00**	105 15.38	103 13.19	105 15.38	95 4.40	107 17.58	**91 0.00**
gdb-18	164	164 0.00	182.5 11.28	177.4 8.17	174.6 6.46	166.4 1.46	167.2 1.95	181 10.37	184 12.20	195 18.90	166 1.22	217 32.32	**164 0.00**
gdb-19	55	55 0.00	71 29.09	55.2 0.36	**55 0.00**	**55 0.00**	55.4 0.73	**55 0.00**	**55 0.00**	61 10.91	**55 0.00**	75 36.36	**55 0.00**
gdb-20	121	121 0.00	132 9.09	124.2 2.64	124.7 3.06	123.2 1.82	123.3 1.90	**123 1.65**	**123 1.65**	**123 1.65**	**123 1.65**	155 28.10	**123 1.65**
gdb-21	156	156 0.00	178 14.10	165.7 6.22	163.1 4.55	160.2 2.69	**159.8 2.44**	172 10.26	160 2.56	162 3.85	160 2.56	201 28.85	160 2.56
gdb-22	200	200 0.00	212.2 6.10	205.8 2.90	205.1 2.55	203.9 1.95	**203.8 1.90**	222 11.00	213 6.50	220 10.00	204 2.00	227 13.50	204 2.00
gdb-23	233	233.4 0.17	251.5 7.94	245.3 5.28	244.3 4.85	242.4 4.03	238.8 2.49	280 20.17	250 7.30	276 18.45	244 4.72	288 23.61	**235 0.86**
Gap [%]	0	0.12	19.03	7.12	6.02	2.87	2.47	7.84	4.90	9.59	2.00	44.81	**1.95**

Table 6
Comparison of solution costs (black) and gap [%] (blue) on the *Eglese* benchmark instances. Lower values indicate better performance. For each instance, the best and second-best results are highlighted in **bold** and underline, respectively, among the constructive heuristic and learning-based methods only. The lower bound and meta-heuristic methods are excluded from ranking. PS denotes path-scanning. PS-A1, PS-A2, and PS-Eff. denote its alt1, alt2, and efficiency variants, respectively.

Inst.	LB	Meta Heuristic					Constructive Heuristic				Learning-based		
		MAENS Tang et al. (2009)	QICA Shang et al. (2018)	MANSP Li et al. (2021)	CARPET Hertz et al. (2000)	MA-ABC Ramamoorthy et al. (2021)	PS Golden et al. (1983)	PS-Eff. Arakaki and Usberti (2019)	PS-A1 Arakaki and Usberti (2019)	PS-A2 Arakaki and Usberti (2019)	AEGT Jia et al. (2025)	DAR Wang et al. (2025b)	DaAM
egl-e1-A	3548	3548 0.00	3548 0.00	3548 0.00	3625 2.17	3590 1.18	3875 9.22	**3681 3.75**	3807 7.30	3850 8.51	4564 28.64	8490 139.29	3804 7.22
egl-e1-B	4498	4498 0.00	4498 0.00	4498 0.00	4532 0.76	4566 1.51	4615 2.60	4646 3.29	4646 3.29	**4567 1.53**	5631 25.19	9373 108.38	4651 3.40
egl-e1-C	5566	5595 0.52	5595 0.52	5595 0.52	5663 1.74	5726 2.87	6039 8.50	6114 9.85	6114 9.85	5980 7.44	6816 22.46	11 080 99.07	**5828 4.71**
egl-e2-A	5018	5018 0.00	5018 0.00	5019 0.02	5233 4.28	5123 2.09	5348 6.58	**5199 3.61**	5207 3.77	5321 6.04	6236 24.27	17 356 245.87	5305 5.72
egl-e2-B	6305	6317 0.19	6314 0.14	6317 0.19	6422 1.86	6438 2.11	6806 7.95	6653 5.52	6647 5.42	**6596 4.62**	7849 24.49	18 686 196.37	6794 7.76
egl-e2-C	8243	8335 1.12	8335 1.12	8335 1.12	8603 4.37	8530 3.48	8909 8.08	8808 6.85	8808 6.85	8966 8.77	9859 19.60	17 432 111.48	**8781 6.53**
egl-e3-A	5898	5898 0.00	5898 0.00	5898 0.00	5907 0.15	6048 2.54	**6099 3.41**	6149 4.26	6465 9.61	6270 6.31	7604 28.93	14 540 146.52	6348 7.63
egl-e3-B	7704	7775 0.92	7775 0.92	7777 0.95	7921 2.82	8033 4.27	**8141 5.67**	8184 6.23	8184 6.23	8189 6.30	9307 20.81	17 261 124.05	8322 8.02
egl-e3-C	10 163	10 292 1.27	10 292 1.27	10 342 1.76	10 805 6.32	10 423 2.56	11 074 8.96	10 821 6.47	10 701 5.29	10 977 8.01	12 118 19.24	18 903 86.00	**10 633 4.62**
egl-e4-A	6408	6456 0.75	6446 0.59	6464 0.87	6489 1.26	6716 4.81	7014 9.46	6975 8.85	6995 9.16	**6920 7.99**	7648 19.35	15 410 140.48	6936 8.24
egl-e4-B	8884	8998 1.28	8975 1.02	9000 1.31	9216 3.74	9404 5.85	9795 10.25	9694 9.12	**9572 7.74**	9655 8.68	10 899 22.68	19 354 117.85	9655 8.68
egl-e4-C	11 427	11 561 1.17	11 567 1.23	11 731 2.66	11 824 3.47	11 775 3.05	12 738 11.47	12 289 7.54	**12 114 6.01**	12 555 9.87	14 082 23.23	20 647 80.69	12 353 8.10
egl-s1-A	5018	5018 0.00	5018 0.00	5018 0.00	5149 2.61	5168 2.99	5600 11.60	5456 8.73	**5388 7.37**	5458 8.77	6393 27.40	12 714 153.37	5445 8.51
egl-s1-B	6384	6388 0.06	6388 0.06	6413 0.45	6641 4.03	6586 3.16	7012 9.84	7009 9.79	7028 10.09	7037 10.23	7807 22.29	9551 49.61	**6642 4.04**
egl-s1-C	8493	8518 0.29	8518 0.29	8518 0.29	8687 2.28	8571 0.92	9326 9.81	9302 9.53	9267 9.11	9457 11.35	9990 17.63	11 427 34.55	**8955 5.44**
egl-s2-A	9824	9895 0.72	9909 0.87	9907 0.84	10 373 5.59	10 455 6.42	**10 897 10.92**	11 099 12.98	11 023 12.20	10 911 11.06	13 431 36.72	43 514 342.94	11 132 13.31
egl-s2-B	12 968	13 147 1.38	13 110 1.10	13 242 2.11	13 495 4.06	13 749 6.02	14 248 9.87	14 225 9.69	14 237 9.79	**14 166 9.24**	16 409 26.53	57 390 342.55	14 330 10.50
egl-s2-C	16 353	16 430 0.47	16 425 0.44	16 437 0.51	17 121 4.70	17 058 4.31	17 955 9.80	**17 705 8.27**	**17 705 8.27**	18 162 11.06	19 742 20.72	53 768 228.80	18 103 10.70
egl-s3-A	10 143	10 257 1.12	10 221 0.77	10 262 1.17	10 541 3.92	10 684 5.33	11 349 11.89	**11 131 9.74**	11 156 9.99	11 390 12.29	13 631 34.39	18 302 80.44	11 360 12.00
egl-s3-B	13 616	13 749 0.98	13 692 0.56	13 786 1.25	14 291 4.96	14 368 5.52	15 158 11.32	14 947 9.78	14 950 9.80	**14 936 9.69**	17 029 25.07	33 112 143.18	15 123 11.07
egl-s3-C	17 100	17 207 0.63	17 214 0.67	17 260 0.94	17 789 4.03	17 980 5.15	18 886 10.44	18 577 8.64	**18 481 8.08**	18 812 10.01	20 698 21.04	39 264 129.61	18 845 10.20
egl-s4-A	12 143	12 341 1.63	12 265 1.00	12 409 2.19	13 036 7.35	13 170 8.46	13 511 11.27	**13 284 9.40**	13 572 11.77	13 658 12.48	16 062 32.27	18 779 54.65	13 781 13.49
egl-s4-B	16 093	16 337 1.52	16 262 1.05	16 416 2.01	16 924 5.16	17 077 6.11	17 985 11.76	**17 626 9.53**	17 686 9.90	17 781 10.49	20 580 27.88	22 745 41.33	17 871 11.05
egl-s4-C	20 375	20 538 0.80	20 505 0.64	20 706 1.62	21 486 5.45	21 500 5.52	22 757 11.69	22 364 9.76	**22 282 9.36**	22 485 10.36	25 484 25.07	27 551 35.22	22 589 10.87
Gap [%]	0	0.70	0.59	0.95	3.63	4.01	9.26	**7.96**	8.18	8.80	24.83	134.68	8.41

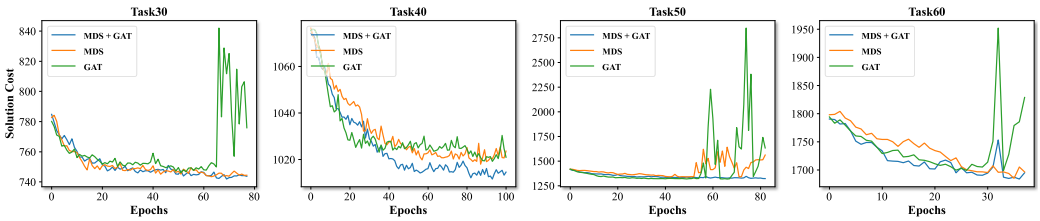


Fig. 3. Convergence trends of different variants of DaAM in reinforcement learning training. The **green** line indicates the model without MDS. The **orange** line indicates the model without GAT. The **blue** line indicates using both MDS and GAT. The line graph shows that the training is smoother and converges faster when using both.

- **GAT** (w/o MDS) removes the MDS-based coordinate features (*i.e.*, the start/end embeddings) from the arc feature vector, while keeping all other static/dynamic features and the overall architecture unchanged (the encoder is still GAT+AM).
- **MDS** (w/o GAT) removes the GAT module. The remaining encoder becomes an attention-only AM encoder that directly processes the arc tokens, consistent with the attention model described in Section 4.1.

The experiment is carried out on Task30, Task40, Task50, and Task60 by comparing the average performance of 1000 instances on each dataset. In the RL stage, we use the policy pre-trained on Task30 for initialization. Table 7 report results as *mean* ± *std* over 4 random seeds for all variants. Removing MDS consistently degrades performance and increases sensitivity to randomness (*e.g.*, Task30: 795.25 ± 34.27 vs. 749.75 ± 6.40; Task60: 1783.50 ± 78.89 vs. 1675.25 ± 32.70). This is expected because without MDS the input representation becomes less informative (only the remaining low-dimensional static/dynamic scalars), making training more sensitive to initialization and data order, which is also consistent with the observation in our training curves (Fig. 3). In contrast, removing GAT causes a smaller but consistent degradation, supporting our hypothesis that GAT mainly helps model structural (arc-arc) interactions in the transformed graph. Although the gap between GAT (w/o MDS) and the full model (MDS+GAT) can be modest on some tasks, the improvement is systematic, as MDS+GAT achieves lower mean costs on Task30/40/50/60.

Fig. 3 shows the convergence trends in these scenes, providing a more concrete view of training stability across scales (Task30-Task60). The full model (MDS+GAT) converges smoothly and maintains a consistently low solution cost with small fluctuations, while the ablated variants exhibit noticeably fewer stable dynamics. In particular, the GAT-only variant shows clear instability with abrupt cost spikes/oscillations in later epochs (*e.g.*, after 60 epochs on Task30 and around 55–75 epochs on Task50, as well as a sharp spike on Task60), indicating occasional divergence from good policies. The MDS-only variant is generally more stable than GAT-only but tends to plateau at a higher cost than the full model. Overall, these curves suggest that MDS and GAT provide complementary inductive biases: MDS supplies direction-aware geometric cues, while GAT captures arc-arc relational structure, and their synergy yields both faster convergence and markedly improved training stability.

5.3. Detailed discussion

Training time analysis. The training time for DaAM is shown in Fig. 4. The '+' symbol indicates the additional training time required at each training stage, building upon the previous model. Furthermore, we evaluate the training time with random initialization for reinforcement learning. It is evident that as the scale of instances increases, the training duration for the current stage also increases. However, this remains within an acceptable range and is considerably less than the training times observed in other tasks. It takes only 0.6 h to train a DaAM for Task 20, while it takes 9.6 h totally to train a DaAM for Task 100.

Table 7
Solution costs of DaAM using GAT, MDS, and GAT+MDS. The best results are indicated in bold.

Variants	Task30	Task40	Task50	Task60
MDS (w/o GAT)	760.75 ± 12.28	1051.25 ± **25.29**	1354.00 ± **20.35**	1716.00 ± **24.29**
GAT (w/o MDS)	795.25 ± 34.27	1062.75 ± 35.19	1394.50 ± 65.30	1783.50 ± 78.89
MDS+GAT	**749.75** ± **6.40**	**1047.75** ± 25.34	**1327.00** ± 25.01	**1675.25** ± 32.70

While reinforcement learning algorithms can start with random strategies, they often require more training time to reach convergence or may fail to converge at all. Therefore, supervised learning is indispensable. As seen, Task 20 can converge in about 2.5 h, but for other task sizes, we spent a significant amount of time training with difficulty in achieving convergence. Additionally, for converging in Task 20, the cost of training with random initialization is higher than that with the combined SL+RL approach. In fact, since supervised learning is only applied on Task20, the SL pre-training remains low.

Inference time analysis. We compare the inference time required for solving 100 CARP instances across datasets Task20 to Task100 datasets using DaAM, MAENS, and PS algorithms, and show the run time in log space. We omit Task30 and Task50 for equal spacing of X-Axis. For datasets ranging from Task200 to Task600, we compare the same metric across PS variants and our method. For our method, we measured the solving time with and without PO. Fig. 5 demonstrates that our method exhibits a significant speed advantage over MAENS, even outperforming variants of PS (Arakaki and Usberti, 2019) in most datasets. In comparison, the consumption time of MAENS increases exponentially as the problem scale increases. Our method efficiently generates paths for large-scale CARP instances by leveraging GPU data-level parallelism and CPU instruction-level parallelism. It is noteworthy that the number of iterations for PS variants is set to only 100 when recording the computation time on instances from Task200 to Task600. This means that if using the official recommended number of iteration, *i.e.*, 1000, the computation time of the PS variants would increase to 10 times the current time cost. By contrast, DaAM demonstrates significantly higher efficiency.

In addition, we found that the time cost of DaAM+PO is generally 6 to 7 times that of DaAM. This is because DaAM can be highly parallelized when running on the GPU, while PO cannot. Therefore, the relative slowdown can look large even when the absolute overhead is small. In our implementation, PO performs a single Split/route-partitioning DP pass with an early break once the accumulated demand exceeds the vehicle capacity, resulting in a predictable and bounded per-instance overhead. Hence, PO is not the runtime bottleneck in practice, and the overall latency remains acceptable.

Effect of PO over constructive heuristics. Since the PO is able to optimize solutions, it is potentially useful for other constructive heuristic algorithms. To verify this point, we further leverage PO to optimize constructive heuristics, as shown in Table 8. Overall, PO consistently enhances the routes obtained by PS across all scales. On Task 20

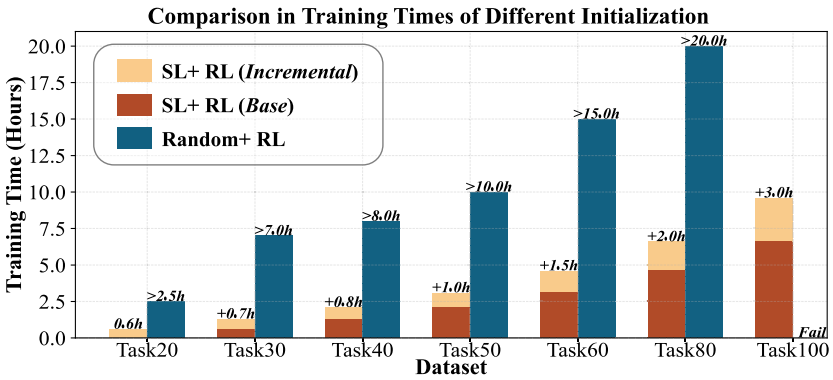


Fig. 4. Training time comparison of models with different initialization. The **blue** bar indicates the time cost with random initialization. In contrast, the two other bars indicates that with supervised learning (SL) based initialization, in which **brown** indicates the time cost of the last training, and **pink** indicates the incremental time cost. The cost of generating MAENS labels is a one-time offline pre-processing step, and it is therefore not included in the training time reported.

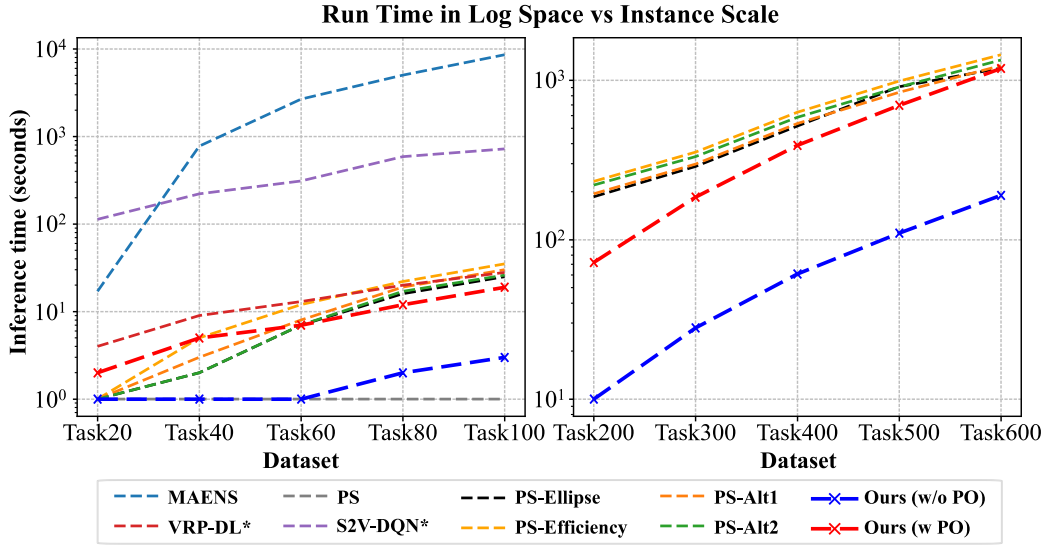


Fig. 5. Run time comparison on Beijing-CARP data set. For each dataset, the total run time of each method on 100 CARP instances is shown. Note that the vertical axis is represented in log space. The **blue** and **red** dashed lines represent the time cost of DaAM without and with PO, respectively. Note that, for *Task20/30/40/50/60/80/100*, the runtimes of PS are *3.8/6.5/9.4/13.5/18.4/30.1/44.6 ms* per 100 CARP instances (with shortest-path distances pre-computed /cached). These sub-second runtimes are compressed near 10^0 and may visually look constant.

Table 8
Performance of path scanning variants using path optimization. ↓ the decrease in Gap [%]. The Gap [%] is measured relative to MAENS over *full precision cost*.

Method	Task20		Task40		Task60		Task80		Task100	
	Cost	Gap [%]	Cost	Gap [%]	Cost	Gap [%]	Cost	Gap [%]	Cost	Gap [%]
MAENS (Tang et al., 2009)	474	0	950	0	1529	0	2113	0	2757	0
PS-Ellipse	519	9.49	1006	5.89	1709	11.77	2299	8.80	3095	12.26
PS-Ellipse + PO	**500**	**5.49** ↓4.00	**998**	**5.05** ↓0.84	**1661**	**8.63** ↓3.14	**2284**	**8.06** ↓0.74	**3045**	**10.45** ↓1.81
PS-Efficiency	514	8.44	1007	6.00	1684	10.14	2282	8.00	3056	10.85
PS-Efficiency + PO	**499**	**5.20** ↓3.24	**996**	**4.84** ↓1.16	**1651**	**7.98** ↓2.16	**2268**	**7.34** ↓0.66	**3025**	**9.72** ↓1.13
PS-Alt1	514	8.44	1007	6.00	1685	10.2	2283	8.04	3057	10.88
PS-Alt1 + PO	**499**	**5.27** ↓3.17	**996**	**4.84** ↓1.16	**1652**	**8.04** ↓2.16	**2269**	**7.38** ↓0.66	**3025**	**9.72** ↓1.16
PS-Alt2	521	9.92	1009	6.21	1720	12.49	2314	9.51	3102	12.51
PS-Alt2 + PO	**499**	**5.27** ↓4.65	**1001**	**5.37** ↓0.84	**1666**	**8.96** ↓3.53	**2295**	**8.61** ↓0.90	**3054**	**10.77** ↓1.74

and Task 60, PO achieves a gap reduction of 2%–5%, while the decrease in the gap is more modest on other scales. Such an experiment demonstrates the feasibility of path optimization on other constructive heuristic methods.

Stability across random seeds. To assess the stability of DaAM, we conduct a multi-seed evaluation on two representative small-scale tasks (Task20 and Task30). Specifically, we train DaAM with four different random seeds, where randomness mainly comes from data shuffling and the optimization process, and report *mean* ± *std* over the runs. As shown in Table 9, DaAM exhibits consistent performance across seeds. On Task20, DaAM (SL+RL) achieves a Gap of $5.38 \pm 0.66\%$, while DaAM (SL+RL+PO) achieves $2.32 \pm 0.75\%$ On Task30, the corresponding gaps

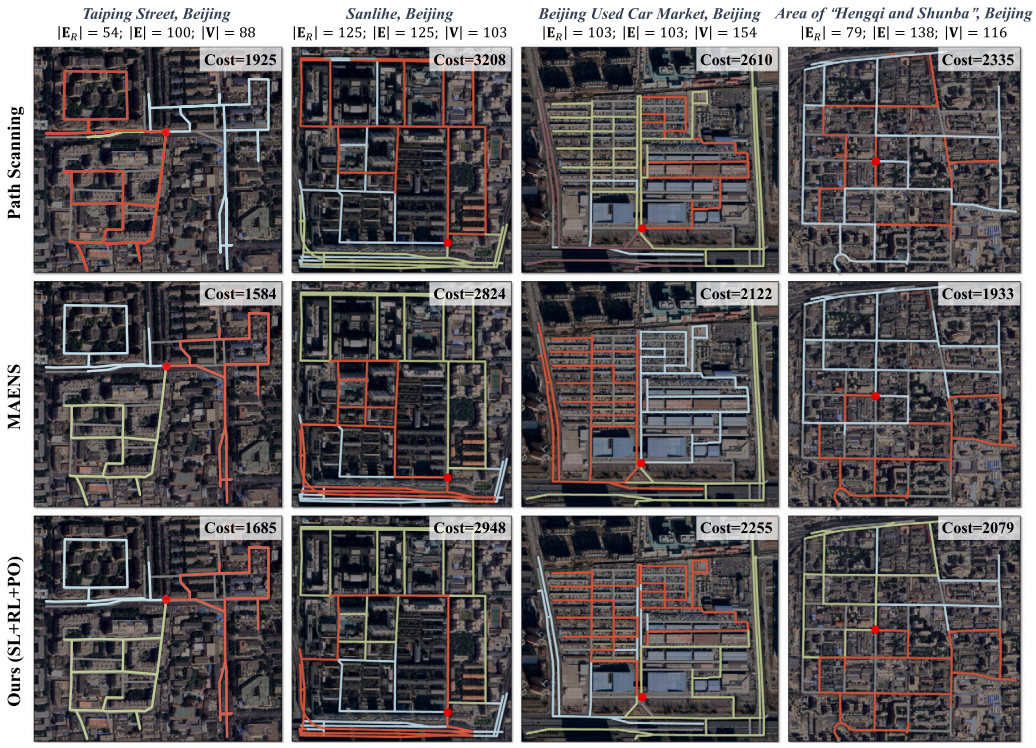


Fig. 6. Qualitative comparison in four real street scenes. The paths are marked in different colors, with gray indicating roads that do not require service and red points indicating depots.

Table 9
Performance on two representative small-scale tasks under four random seeds (mean ± std).

Method	Task20		Task30	
	Cost	Gap [%]	Cost	Gap [%]
MAENS	474	0.00%	706	0.00%
DaAM (SL+RL)	499.50 ± 3.11	5.38 ± 0.66	749.75 ± 6.40	6.20 ± 0.91
DaAM (SL+RL+PO)	485.00 ± 3.56	2.32 ± 0.75	727.50 ± 2.38	3.05 ± 0.34

are 6.20 ± 0.91% and 3.05 ± 0.34%, respectively. These results suggest that the improvements brought by PO are stable under different random seeds on these tasks, and the reinforcement learning process is stable enough.

5.4. Solution visualization and qualitative analysis

To complement the cost-based comparison, we provide qualitative visualizations of representative solutions produced by different methods, PS (Golden et al., 1983), MAENS, and our DaAM. These examples are intended to illustrate route partition patterns, such as spatial locality and inter-route overlap, rather than to serve as a systematic quantitative evaluation over the full test set.

Fig. 6 presents representative qualitative examples together with the corresponding scene information. We observe that DaAM, trained with MAENS-generated supervision, often produces route structures similar to those of MAENS. In the smallest map (1st column), the red and blue routes generated by PS are relatively long and show noticeable overlap with the green route. By comparison, both MAENS and DaAM produce solutions with clearer spatial separation, partitioning the scene into more localized regions. Such visually less-overlapping route layouts may be more attractive in practice, although they do not necessarily imply lower total cost. In the larger map (2nd column), where all edges are required edges, MAENS still exhibits strong spatial locality, and the solution generated by DaAM remains visually close to it, while PS shows more overlap between the blue and green routes. Again, this difference mainly reflects route organization and locality. In the 3rd column, which corresponds to a map with more nodes, PS produces one more route than MAENS and DaAM, and the route layout appears less localized. Although DaAM attains a lower solution cost in this case, its route partition is visually somewhat less compact than that of MAENS, indicating that better spatial separation and lower cost do not always coincide. In the setting with sparse required edges (4th column), PS produces two routes that both traverse a large portion of the map, whereas MAENS yields two more clearly separated routes. DaAM generates one additional route in this case, but its routes remain relatively independent and exhibit limited overlap overall.

6. Conclusion and limitations

In this paper, we propose a learning-based CARP solver that competes with state-of-the-art meta-heuristics. Firstly, we encode the potential serving direction of edges into embeddings, ensuring that edge directionality is considered in decision-making. Secondly, we present a supervised reinforcement learning approach that effectively learns policies to solve CARP. With these contributions, DaAM outperforms the PS-based constructive heuristics we tested (PS and several widely used PS variants) on the CARP-Beijing benchmark. The proposed direction-aware embedding and learning scheme may also be extended to other arc-routing variants, such as capacitated arc routing with time windows and multi-depot settings, which we leave for future work.

Limitations and Future Work. DaAM is a fast constructive neural solver with a lightweight Split-based post-optimizer (PO). A limitation is that PO only optimizes route partitioning (depot insertions) for a decoded arc tour and does not perform broader local improvement. In addition, although DaAM generalizes well across scales, performance may degrade under larger distribution shifts in topology, demand patterns, or cost structures beyond our generator.

Future work will focus on large-scale CARP. We plan to benchmark and integrate stronger large-scale heuristics/metaheuristics (*e.g.*, decomposition/ divide-and-conquer and hybrid genetic/memetic search) with DaAM, using the neural policy for fast arc-level construction or candidate generation and scalable heuristics for efficient refinement under a fixed time budget. We will also broaden training distributions and accelerate inference via lightweight architectures, distillation, and quantization to further improve the quality–time trade-off. In addition, due to implementation constraints, DaAM can only be trained on a set of instances with the same amount of required edge. This prevents us from training on instance data of multi-scale data simultaneously, thus making a completely fair comparison with SPLICE (Ramamoorthy and Syrotiuk, 2024) impossible. To further address the generalization problem, we plan to extend DaAM to multi-scale training strategy. Finally, a more systematic quantitative evaluation of route compactness and locality is an interesting direction for future work. For example, one may introduce per-instance metrics based on route overlap or spatial dispersion, and summarize them across the benchmark to complement the current qualitative visualization.

CRediT authorship contribution statement

Feng Xue: Writing – original draft, Software, Formal analysis, Data curation, Conceptualization. **Runze Guo:** Writing – original draft, Validation, Software, Resources, Data curation. **Anlong Ming:** Writing – review & editing, Supervision, Project administration, Funding acquisition, Conceptualization. **Nicu Sebe:** Supervision, Conceptualization.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgment

This work was supported by the Funds for Creative Research Groups of China under Grant (no. 61921003).

Appendix. Performance on the chinese postman problem

We evaluated DaAM on the Chinese Postman Problem (CPP) as a special case of CARP by removing the capacity-induced multi-route effect (*i.e.*, enforcing a single tour/sufficiently large capacity so no splitting is required) while requiring all tasks to be served. Table A.10 reports the results on the gdb instances. As shown, DaAM achieves the optimal CPP cost on most instances (0.00% gap), and the remaining cases are only marginally suboptimal (within 1.60%–2.02%). Moreover, DaAM (SL+RL) and DaAM (SL+RL+PO) yield identical costs in this CPP setting, which is expected because PO is a Split-style route-partitioning procedure and becomes inactive when the solution reduces to a single route (no depot-return insertion is needed). These results indicate that DaAM remains effective on the polynomial-time CPP special case, while PO mainly benefits the general CARP setting where multi-route partitioning under capacity constraints is required.

Data availability

No data was used for the research described in the article.

Table A.10
Results on the gdb instances degraded to the Chinese Postman Problem (CPP).

Instance name	Optimal (CPP)	DaAM (SL+RL+PO)		DaAM (SL+RL)	
		Cost	Gap [%]	Cost	Gap [%]
gdb1.dat	294	294	0.00	294	0.00
gdb2.dat	315	315	0.00	315	0.00
gdb3.dat	259	259	0.00	259	0.00
gdb4.dat	266	266	0.00	266	0.00
gdb5.dat	346	353	2.02	353	2.02
gdb6.dat	279	279	0.00	279	0.00
gdb7.dat	304	304	0.00	304	0.00
gdb8.dat	250	254	1.60	254	1.60
gdb9.dat	247	252	2.02	252	2.02
gdb10.dat	275	275	0.00	275	0.00
gdb11.dat	387	387	0.00	387	0.00
gdb12.dat	384	384	0.00	384	0.00
gdb13.dat	520	520	0.00	520	0.00
gdb14.dat	96	96	0.00	96	0.00
gdb15.dat	56	56	0.00	56	0.00
gdb16.dat	125	125	0.00	125	0.00
gdb17.dat	91	91	0.00	91	0.00
gdb18.dat	158	158	0.00	158	0.00
gdb19.dat	55	55	0.00	55	0.00
gdb20.dat	121	121	0.00	121	0.00
gdb21.dat	154	154	0.00	154	0.00
gdb22.dat	196	196	0.00	196	0.00
gdb23.dat	223	223	0.00	223	0.00

References

Agarwal, S., Akella, S., 2024. Line coverage with multiple robots: Algorithms and experiments. IEEE Trans. Robotics (TRO) 40, 1664–1683.

Arakaki, R.K., Usberti, F.L., 2019. An efficiency-based path-scanning heuristic for the capacitated arc routing problem. Comput. Oper. Res. (COR) 103, 288–295.

Belenguer, J.-M., Benavent, E., Lacomme, P., Prins, C., 2006. Lower and upper bounds for the mixed capacitated arc routing problem. Comput. Oper. Res. 33 (12), 3363–3383.

Bello, I., Pham, H., Le, Q.V., Norouzi, M., Bengio, S., 2017. Neural combinatorial optimization with reinforcement learning. In: International Conference on Learning Representations. ICLR.

Berahman, M., Karalakou, A., Rostami-Shahrbabaki, M., Bogenberger, K., 2025. Multi-task lane-free driving strategy for connected and automated vehicles: A multi-agent deep reinforcement learning approach. Eng. Appl. Artif. Intell. (EAAI) 154, 110797.

Beullens, P., Muyldermans, L., Cattrysse, D., Van Oudheusden, D., 2003. A guided local search heuristic for the capacitated arc routing problem. European J. Oper. Res. 147 (3), 629–643. http://dx.doi.org/10.1016/S0377-2217(02)00334-X.

Bode, C., Irnich, S., 2012. Cut-first branch-and-price-second for the capacitated arc-routing problem. Oper. Res. 60 (5), 1167–1182. http://dx.doi.org/10.1287/opre.1120.1079.

Brandão, J., Eglese, R., 2008. A deterministic tabu search algorithm for the capacitated arc routing problem. Comput. Oper. Res. 35 (4), 1112–1126. http://dx.doi.org/10.1016/j.cor.2006.07.007.

Campuzano, G., Lalla-Ruiz, E., Mes, M., 2025. The two-tier multi-depot vehicle routing problem with robot stations and time windows. Eng. Appl. Artif. Intell. (EAAI) 147, 110258.

Chang, Y., Xue, F., Sheng, F., Liang, W., Ming, A., 2022. Fast road segmentation via uncertainty-aware symmetric network. In: International Conference on Robotics and Automation. ICRA, pp. 11124–11130.

Chen, Y., Hao, J.-K., Glover, F., 2016. A hybrid metaheuristic approach for the capacitated arc routing problem. Eur. J. Oper. Res. (EJOR) 253 (1), 25–39.

Chen, D., Lin, Y., Li, W., Li, P., Zhou, J., Sun, X., 2020. Measuring and relieving the over-smoothing problem for graph neural networks from the topological view. In: AAAI Conference on Artificial Intelligence. AAAI.

Corberán, Á., Laporte, G., 2015. Arc Routing: Problems, Methods, and Applications. SIAM.

Corberán, Á., Prins, C., 2010. Recent results on arc routing problems: An annotated bibliography. Networks 56 (1), 50–69. http://dx.doi.org/10.1002/net.20347.

Dai, H., Dai, B., Song, L., 2016. Discriminative embeddings of latent variable models for structured data. In: International Conference on Machine Learning. ICML.

Debarshi, S., Sundaram, S., Sundararajan, N., 2022. Robust EMRAN-aided coupled controller for autonomous vehicles. Eng. Appl. Artif. Intell. (EAAI) 110, 104717.

Duan, H., Xiao, K., Chen, C., Xu, M., Yao, X., 2025. REINFORCE with bound-guided gradient estimator for the traveling salesman problem toward scale generalization. Eng. Appl. Artif. Intell. (EAAI) 141, 109819.

Easton, K., Burdick, J., 2005. A coverage algorithm for multi-robot boundary inspection. In: IEEE International Conference on Robotics and Automation. ICRA, IEEE.

Ebadinezhad, S., 2020. DEACO: Adopting dynamic evaporation strategy to enhance ACO algorithm for the traveling salesman problem. Eng. Appl. Artif. Intell. (EAAI) 92, 103649.

Fan, R., Ozgunalp, U., Wang, Y., Liu, M., Pitas, I., 2022. Rethinking road surface 3-D reconstruction and pothole detection: From perspective transformation to disparity map segmentation. IEEE Trans. Cybern. (T-CYB) 52 (7), 5799–5808.

Fu, H., Liu, W., Wang, Q., Li, Y., Lin, R., 2025. Intrinsic-motivation multi-robot social formation navigation with coordinated exploration. Eng. Appl. Artif. Intell. (EAAI) 159, 111740.

Golden, B.L., DeArmon, J.S., Baker, E.K., 1983. Computational experiments with algorithms for a class of routing problems. Comput. Oper. Res. (COR) 10 (1), 47–59.

Golden, B.L., Wong, R.T., 1981. Capacitated arc routing problems. Networks 11 (3), 305–315.

Haklay, M., Weber, P., 2008. Openstreetmap: User-generated street maps. IEEE Pervasive Comput. 7 (4), 12–18.

Hamilton, W., Ying, Z., Leskovec, J., 2017. Inductive representation learning on large graphs. In: Advances in Neural Information Processing Systems. NeurIPS.

Hertz, A., Laporte, G., Mittaz, M., 2000. A tabu search heuristic for the capacitated arc routing problem. Oper. Res. 48 (1), 129–135.

Hong, W., Liu, T., 2022. Faster capacitated arc routing: A sequence-to-sequence approach. IEEE Access 10, 4777–4785.

Huo, F., Mao, J., Sun, C., Li, Z., Zhang, B., 2025. Deep reinforcement learning of group consciousness for multi-robot pathfinding. Eng. Appl. Artif. Intell. (EAAI) 155, 110978.

Jia, Y.-H., Zheng, Q., Wang, Y., Mei, Y., Chen, W.-N., Lin, Z., 2025. A neural solver with traversal-based feature representation and adjacent attention for capacitated arc routing problem. IEEE Trans. Intell. Transp. Syst. http://dx.doi.org/10.1109/TITS.2025.3611948.

Khalil, E., Dai, H., Zhang, Y., Dilkina, B., Song, L., 2017. Learning combinatorial optimization algorithms over graphs. In: Advances in Neural Information Processing Systems. NeurIPS.

Kingma, D., Ba, J., 2015. Adam: A method for stochastic optimization. In: International Conference on Learning Representations. ICLR.

Kipf, T.N., Welling, M., 2017. Semi-supervised classification with graph convolutional networks. In: International Conference on Learning Representations. ICLR.

Kool, W., van Hoof, H., Welling, M., 2019. Attention, learn to solve routing problems!. In: International Conference on Learning Representations. ICLR.

Krasnogor, N., Smith, J., 2005. A tutorial for competent memetic algorithms: model, taxonomy, and design issues. IEEE Trans. Evol. Comput. (TEVC) 9 (5), 474–488.

Kwon, Y.-D., Choo, J., Kim, B., Yoon, I., Gwon, Y., Min, S., 2020. Pomo: Policy optimization with multiple optima for reinforcement learning. Adv. Neural Inf. Process. Syst. 33, 21188–21198.

Kwon, Y.-D., Choo, J., Yoon, I., Park, M., Park, D., Gwon, Y., 2021. Matrix encoding networks for neural combinatorial optimization. Adv. Neural Inf. Process. Syst. 34, 5138–5149.

Labadie, N., Prins, C., Prodhon, C., 2016. Simple heuristics and local search procedures. In: Metaheuristics for Vehicle Routing Problems. Vol. 3, Wiley Online Library, pp. 15–37.

Li, L.Y., Eglese, R.W., 1996. An interactive algorithm for vehicle routeing for winter—gritting. J. Oper. Res. Soc. 47 (2), 217–228.

Li, H., Li, G., 2019. Learning to solve capacitated arc routing problems by policy gradient. In: IEEE Congress on Evolutionary Computation. CEC.

Li, X., Zhang, S., 2024. Discrete artificial bee colony algorithm with fixed neighborhood search for traveling salesman problem. Eng. Appl. Artif. Intell. (EAAI) 131, 107816.

Li, R., Zhao, X., Zuo, X., Yuan, J., Yao, X., 2021. Memetic algorithm with non-smooth penalty for capacitated arc routing problem. Knowl.-Based Syst. 220, 106957.

Lin, X., Huang, J., Zhang, B., 2025. A velocity adaptive steering control strategy of autonomous vehicles based on double deep Q-learning network with varied agents. Eng. Appl. Artif. Intell. (EAAI) 139, 109655.

Liu, F., Lin, X., Wang, Z., Zhang, Q., Xialiang, T., Yuan, M., 2024. Multi-task learning for routing problem with cross-problem zero-shot generalization. In: ACM SIGKDD Conference on Knowledge Discovery and Data Mining. KDD.

Mei, Y., Li, X., Yao, X., 2014. Cooperative coevolution with route distance grouping for large-scale capacitated arc routing problems. IEEE Trans. Evol. Comput. 18 (3), 435–449. http://dx.doi.org/10.1109/TEVC.2013.2281503.

Mnih, V., Badia, A.P., Mirza, M., Graves, A., Lillicrap, T., Harley, T., Silver, D., Kavukcuoglu, K., 2016. Asynchronous methods for deep reinforcement learning. In: International Conference on Machine Learning. ICML, pp. 1928–1937.

Mnih, V., Kavukcuoglu, K., Silver, D., Rusu, A.A., Veness, J., Bellemare, M.G., Graves, A., Riedmiller, M., Fidjeland, A.K., Ostrovski, G., et al., 2015. Human-level control through deep reinforcement learning. Nature 518 (7540), 529–533.

Nazari, M., Oroojlooy, A., Snyder, L., Takác, M., 2018. Reinforcement learning for solving the vehicle routing problem. In: Advances in Neural Information Processing Systems. NeurIPS.

Nguyen, V.Q., Nguyen, Q.C., Dang, T.H., Hy, T.-S., 2025. Hybridising reinforcement learning and heuristics for hierarchical directed arc routing problems. arXiv:2501.00852, Preprint, arXiv:2501.00852v1.

Oliveira, D.F., Martins, M.S.E., Sousa, J.M.C., Vieira, S.M., Figueira, J.R., 2025. Divide-and-conquer initialization and mutation operators for the large-scale mixed capacitated arc routing problem. European J. Oper. Res. 321 (2), 383–396. http://dx.doi.org/10.1016/j.ejor.2024.09.043.

Pearn, W.L., 1989. Approximate solutions for the capacitated arc routing problem. Comput. Oper. Res. 16 (6), 589–600.

Ramamoorthy, M., Forrest, S., Syrotiuk, V.R., 2021. MA-ABC: a memetic algorithm optimizing attractiveness, balance, and cost for capacitated arc routing problems. In: The Genetic and Evolutionary Computation Conference. GECCO.

Ramamoorthy, M., Syrotiuk, V.R., 2024. Learning heuristics for arc routing problems. Intell. Syst. Appl. (ISWA) 21, 200300.

Rennie, S.J., Marcheret, E., Mroueh, Y., Ross, J., Goel, V., 2017. Self-critical sequence training for image captioning. In: IEEE Conference on Computer Vision and Pattern Recognition. CVPR.

Santos, L., Coutinho-Rodrigues, J., Current, J.R., 2009. An improved heuristic for the capacitated arc routing problem. Comput. Oper. Res. (COR) 36 (9), 2632–2637.

Schulman, J., Wolski, F., Dhariwal, P., Radford, A., Klimov, O., 2017. Proximal policy optimization algorithms. arXiv preprint arXiv:1707.06347.

Shang, R., Du, B., Dai, K., Jiao, L., Esfahani, A.M.G., Stolkin, R., 2018. Quantum-inspired immune clonal algorithm for solving large-scale capacitated arc routing problems. Memetic Comput. 10, 81–102.

Sun, R., Zheng, Z., Wang, Z., 2024. Learning encodings for constructive neural combinatorial optimization needs to regret. In: Proceedings of the AAAI Conference on Artificial Intelligence. Vol. 38, pp. 20803–20811.

Sutskever, I., Vinyals, O., Le, Q.V., 2014. Sequence to sequence learning with neural networks. In: Advances in Neural Information Processing Systems. NeurIPS.

Tang, K., Mei, Y., Yao, X., 2009. Memetic algorithm with extended neighborhood search for capacitated arc routing problems. IEEE Trans. Evol. Comput. (TEVC) 13 (5), 1151–1166.

Tang, M., Mei, Y., Zhang, X., Tang, K., Yao, X., 2017. A scalable approach to capacitated arc routing problems based on hierarchical decomposition. IEEE Trans. Cybern. 47 (11), 3925–3937. http://dx.doi.org/10.1109/TCYB.2016.2636825.

Tran, C.D., Hy, T.S., 2023. Graph attention-based deep reinforcement learning for solving the Chinese postman problem with load-dependent costs. arXiv preprint arXiv:2310.15516.

Ulusoy, G., 1985. The fleet size and mix problem for capacitated arc routing. European J. Oper. Res. 22 (3), 329–337.

Veličković, P., Cucurull, G., Casanova, A., Romero, A., Liò, P., Bengio, Y., 2018. Graph attention networks. In: International Conference on Learning Representations. ICLR.

Vidal, T., 2017. Node, edge, arc routing and turn penalties: Multiple problems—One neighborhood extension. Oper. Res. 65 (4), 992–1010. http://dx.doi.org/10.1287/opre.2017.1618.

Vinyals, O., Fortunato, M., Jaitly, N., 2015. Pointer networks. In: Advances in Neural Information Processing Systems. NeurIPS.

Wang, Y., Gou, M., Luo, S., Fan, J., Wang, H., 2025a. The multi-depot pickup and delivery vehicle routing problem with time windows and dynamic demands. Eng. Appl. Artif. Intell. (EAAI) 139, 109700.

Wang, Y., Jia, Y.-H., Chen, W.-N., Mei, Y., 2025b. Distance-aware attention reshaping for enhancing generalization of neural solvers. IEEE Trans. Neural Networks Learn. Syst. 36 (10), 18900–18914. http://dx.doi.org/10.1109/TNNLS.2025.3588209.

Williams, R.J., 1992. Simple statistical gradient-following algorithms for connectionist reinforcement learning. Mach. Learn. 8, 229–256.

Wøhlk, S., 2018. A fast heuristic for large-scale capacitated arc routing problems. J. Oper. Res. Soc. 69 (12), 1877–1887. http://dx.doi.org/10.1080/01605682.2017.1415648.

Wu, Z., Pan, S., Chen, F., Long, G., Zhang, C., Philip, S.Y., 2020. A comprehensive survey on graph neural networks. IEEE Trans. Neural Networks Learn. Syst. (TNNLS) 32 (1), 4–24.

Wu, F., Souza, A., Zhang, T., Fifty, C., Yu, T., Weinberger, K., 2019. Simplifying graph convolutional networks. In: International Conference on Machine Learning. ICML.

Xue, F., Chang, Y., Wang, T., Zhou, Y., Ming, A., 2024. Indoor obstacle discovery on reflective ground via monocular camera. Int. J. Comput. Vis. (IJCV) 132 (3), 987–1007.

Xue, F., Chang, Y., Xu, W., Liang, W., Sheng, F., Ming, A., 2025. Evidence-based real-time road segmentation with RGB-D data augmentation. Trans. Intell. Transp. Syst. (T-its) 26 (2), 1482–1493.

Xue, P., Pi, D., Wan, C., Yang, C., Xie, B., Wang, H., Wang, X., Yin, G., 2024. Collaborative planning and control of heterogeneous multi-ground unmanned platforms. Eng. Appl. Artif. Intell. (EAAI) 136, 108968.

Zhang, Y., Mei, Y., Zhang, B., Jiang, K., 2019. Divide-and-conquer large scale capacitated arc routing problems with route cutting off decomposition. arXiv:1912.12667.